

DOCUMENTACIÓN TÉCNICA

DataLegal

Guía de instalación, arquitectura, operación del código y despliegue en AWS

Audiencia: Personas desarrolladoras, QA y revisoras del código

Repositorio: <https://github.com/Jonthz/datalegal>

Base revisada: rama main · commit 291f533

Fecha de corte: 14 de agosto de 2026

Alcance: descarga, configuración, ejecución local, arquitectura, código, pruebas y AWS

Clasificación: uso interno del equipo DataLegal

Resumen verificado: La solución es un SaaS multi-tenant de cumplimiento LOPDP con 156 endpoints, 33 modelos persistentes y 33 rutas de interfaz. La suite actual aprobó 871 pruebas con 93 % de cobertura del paquete app; TypeScript y el build del frontend también finalizaron sin errores.

El recorrido parte de un clon limpio, explica Docker Compose, ejecución manual y el despliegue AWS observado; después sirve como referencia del código. No contiene contraseñas, tokens, cadenas de conexión ni valores secretos.

Evidencia: inventario AST sobre backend/app y frontend/src; ejecución local de pytest, coverage, TypeScript y Vite; artefactos deploy/aws y verificación HTTP de la URL CloudFront.

1. Propósito, alcance y modo de lectura

DataLegal centraliza actividades de tratamiento, activos de información, riesgos, DPIA, derechos ARCO, incidentes, consentimientos, documentos legales, auditoría, capacitación y reportes para organizaciones sujetas a la Ley Orgánica de Protección de Datos Personales de Ecuador. La arquitectura separa cuentas de plataforma de cuentas de tenant y aplica aislamiento lógico por tenant_id.

1.1 Convenciones

Verificado: comportamiento o estructura comprobados en archivos concretos o mediante comandos ejecutados.

Según documentación del repo: dato declarado en README o comentarios, contrastado cuando fue posible con el código.

Recomendación: mejora propuesta; no describe una capacidad ya implementada.

Las rutas y archivos se expresan relativos a la raíz del repositorio datalegal/.

Base reproducible: Los comandos de inicio usan <https://github.com/Jonthz/datalegal> y la rama main. Después de clonar, registrar git rev-parse --short HEAD: el documento se verificó sobre 291f533 y una revisión posterior puede producir inventarios diferentes.

1.2 Mapa del documento

Secciones	Contenido
2–4	Resumen técnico, arquitectura, repositorio y stack
5–8	Arranque, backend, seguridad, multi-tenancy y datos
9	Inventario completo de la API REST
10	Frontend React/Vite, rutas, cliente API e i18n
11–13	Datos mock, entorno local y variables de entorno
14	Instalación y despliegue verificado en AWS
15–20	Pruebas, diagnóstico, riesgos, mantenimiento, soporte y contribución
21–23	Glosario y apéndices de referencia

La jerarquía Heading 1/2/3 permite navegar desde el panel de navegación de Word.

2. Resumen técnico ejecutivo

Dimensión	Resultado verificado
Producto	SaaS de cumplimiento LOPDP para DPO, auditoría y equipos de cumplimiento.
Backend	FastAPI 0.115+, SQLAlchemy 2, Pydantic Settings, JWT, TOTP, Alembic.
Frontend	React 18, TypeScript 5, Vite 6, Tailwind 3, Axios, react-i18next.
Persistencia	PostgreSQL 16 con Docker Compose y en AWS/EC2; SQLite para ejecución manual y pruebas.
Superficie	156 endpoints REST en 29 routers; 33 tablas/modelos; 101 clases de esquema.
Interfaz	33 rutas: 2 públicas y 31 protegidas; carga diferida por página.
Calidad comprobada	871 pruebas aprobadas; cobertura total app 93 %; build Vite exitoso.
Instalación	Clon Git → variables → Compose/manual → QA → despliegue CloudFront + EC2.

2.1 Fortalezas observadas

Aislamiento de lectura/escritura aplicado de forma consistente en la mayoría de routers mediante `get_current_tenant_id` y filtros `tenant_id`.

Configuración tipada por entorno, con documentación local y valores sensibles fuera del repositorio.

MFA TOTP con secreto cifrado en reposo, logout de cuenta, rate limiting y control de inactividad en servidor.

Matriz RBAC estática más roles personalizados por tenant; permisos de plataforma separados.

Auditoría append-only usada en operaciones sensibles y exportable a CSV.

Amplia suite backend y cobertura alta de API, modelos, esquemas y lógica de negocio.

Rutas frontend reales para todos los módulos registrados; no quedan placeholders conectados.

Despliegue AWS documentado con CloudFront como único ingreso público y tres contenedores aislados en EC2.

2.2 Prioridades para un clon reproducible

Prioridad	Acción
P0	Retirar y rotar el valor de <code>.env.render</code> rastreado; conservar únicamente una plantilla sin secretos.
P0	Publicar las migraciones pendientes y confirmar que alembic heads devuelve una sola revisión en un clon limpio.
P0	Revisar y versionar <code>deploy/aws</code> y <code>docs/deployment-aws.md</code> : hoy existen solo como cambios locales no rastreados.

Prioridad	Acción
P1	Versionar package-lock.json: docker compose usa npm ci y falla en un clon limpio sin lockfile.
P1	Añadir CI de pytest/cobertura y build frontend; el workflow actual ejecuta solo Pylint.
P1	Actualizar README: sus cifras de pruebas ya no coinciden con la suite observada.
P2	Revisar JWT en localStorage, referencias entre tenants y almacenamiento del rate limiter.

Fuentes: README.md, .gitignore, .github/workflows/pylint.yml, migraciones Alembic, package.json y git status.

3. Arquitectura de la solución

La solución sigue una arquitectura web de tres capas. El navegador ejecuta una SPA; la SPA consume una API REST del mismo origen bajo /api/v1; FastAPI ejecuta reglas de autenticación, autorización y negocio; SQLAlchemy persiste el estado. Servicios como auditoría, reportes y alertas atraviesan varios dominios.

Arquitectura lógica de DataLegal

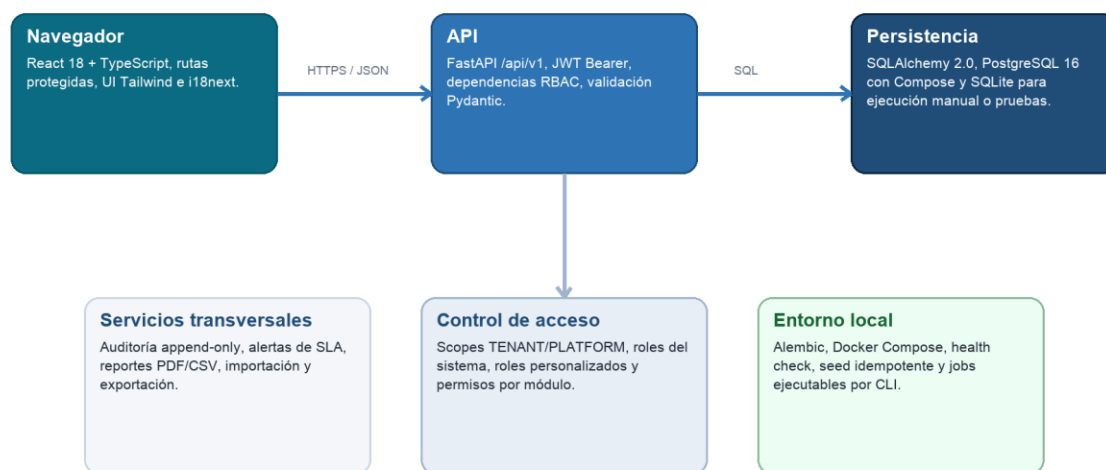


Figura 1. Capas lógicas y servicios transversales.

3.1 Responsabilidades por capa

Capa	Ubicación	Responsabilidad
Presentación	frontend/src	rutas, estado de autenticación, formularios, tablas, estados UI y descargas
API	backend/app/api/v1	contrato HTTP, dependencias, validación, autorización y orquestación
Dominio/datos	backend/app/models, schemas, core, services	persistencia, reglas compartidas, cifrado, SLA y alertas
Entorno local	docker-compose.yml, backend/alembic, scripts	servicios, migraciones, seed, jobs y diagnóstico
Producción AWS	deploy/aws, docs/deployment-aws.md	CloudFront, EC2, SSM, Nginx, contenedores y verificación

3.2 Límites de confianza

El navegador no es autoridad de permisos: ProtectedRoute mejora UX, pero el backend vuelve a validar cada acción.

El JWT identifica al usuario; el tenant efectivo se toma del usuario cargado desde base, no de un parámetro arbitrario del cliente.

Los endpoints PLATFORM exigen account_scope=PLATFORM y permisos explícitos; los endpoints TENANT rechazan cuentas de plataforma.

La base contiene múltiples tenants en tablas compartidas; el aislamiento es lógico, no por esquema o base independiente.

Con Docker Compose, Vite, FastAPI y PostgreSQL son servicios separados; el proxy /api del servidor Vite evita configurar una URL absoluta en la SPA.

En AWS, CloudFront termina HTTPS para el usuario y usa EC2 como origen HTTP restringido; Nginx conserva la API y la SPA bajo el mismo origen.

Fuentes: frontend/src/components/ProtectedRoute.tsx, frontend/vite.config.ts, backend/app/api/deps.py, docker-compose.yml y deploy/aws/deploy-datalegal.sh.

4. Composición del repositorio y stack

4.1 Árbol funcional

```

datalegal/
├── backend/
│   ├── app/api/v1/    routers REST por módulo
│   ├── app/core/     configuración, seguridad, permisos, SLA, middleware
│   ├── app/db/       engine, sesiones y seed de desarrollo
│   ├── app/models/   33 modelos/tablas SQLAlchemy
│   ├── app/schemas/ contratos Pydantic de entrada/salida
│   ├── app/services/ escaneo de alertas críticas
│   ├── alembic/      migraciones y entorno
│   ├── scripts/      backup y escaneo programable
│   └── tests/        suite pytest
├── frontend/
│   ├── src/api/      cliente Axios por dominio
│   ├── src/components/ shell, navegación y UI compartida
│   ├── src/pages/    páginas de rutas públicas/protegidas
│   ├── src/routes/   registro y permisos de navegación
│   ├── src/types/    tipos por dominio
│   └── src/i18n/     traducciones
├── docs/             planes, diseño y revisiones
├── .github/workflows/ controles automatizados del repositorio
├── README.md         panorama funcional y stack
├── docker-compose.yml entorno local de tres servicios
└── backend/.env.example nombres de configuración local
  
```

Conteo verificado: 224 archivos de código/configuración seleccionados y 38.588 líneas en `backend/app`, `backend/tests` y `frontend/src`.

4.2 Tamaño aproximado

Área	Líneas	Lenguajes
Backend de aplicación	11.402	Python
Pruebas backend	5.532	Python
Frontend	21.654	TS/TSX/CSS/JSON
Total seleccionado	38.588	224 archivos

4.3 Dependencias principales

Tecnología	Rango/versión	Uso
Python	>=3.12	runtime backend
FastAPI	>=0.115	API REST
SQLAlchemy	>=2.0	ORM síncrono
Alembic	>=1.13	migraciones
Pydantic Settings	>=2.4	variables de entorno

Tecnología	Rango/versión	Uso
python-jose	>=3.3	JWT HS256
passlib + bcrypt	bcrypt <5	hash de contraseñas
PyOTP + cryptography	>=2.9 / >=42	TOTP y Fernet
slowapi	>=0.1.9	rate limiting
fpdf2	>=2.7.9	PDF
React	18.3	UI
TypeScript	5.7	tipado
Vite	6.3	build/dev server
Tailwind CSS	3.4	estilos
Axios	1.7	cliente HTTP
i18next	23.16	internacionalización

Reproducibilidad: backend/uv.lock sí está versionado, pero el Dockerfile instala con pip install -e . y usa rangos. frontend/package-lock.json está ignorado y no versionado, aunque Docker Compose ejecuta npm ci. Conviene alinear ambos pipelines con lockfiles versionados.

Fuentes: backend/pyproject.toml, backend/uv.lock, frontend/package.json, .gitignore, backend/Dockerfile y docker-compose.yml.

5. Descarga del repositorio y puesta en marcha local

Este capítulo es el recorrido mínimo y reproducible para una persona que recibe únicamente la URL Git. Los apartados posteriores explican el código en profundidad; aquí se prioriza obtener un clon identificable, preparar configuración local sin divulgar secretos y confirmar que interfaz, API y base responden.

5.1 Clonar, entrar y registrar la revisión

```
git clone https://github.com/Jonthz/datalegal.git
cd datalegal
git branch --show-current
git rev-parse --short HEAD
git status --short
sed -n '1,220p' README.md
sed -n '1,220p' backend/.env.example
```

Punto de referencia: La revisión documental usa main en 291f533. Si el hash del clon es distinto, conservar ese hash en la evidencia y volver a comprobar migraciones, dependencias, endpoints y pruebas.

5.2 Requisitos locales

Ruta	Requisito	Uso
Común	Git y terminal	clonar, inspeccionar estado y crear ramas
Opción Compose	Docker Engine/Desktop + Docker Compose v2	levantar PostgreSQL, FastAPI y Vite
Opción manual	Python 3.12	backend, Alembic, seed y pytest
Opción manual	Node.js 20 + npm	frontend Vite, typecheck y build
Base manual	SQLite incluido o PostgreSQL 16	SQLite es el valor local por defecto; PostgreSQL es opcional

```
git --version
docker --version
docker compose version
python3.12 --version
node --version
npm --version
```

5.3 Preparar variables sin copiar secretos

Leer README.md y backend/.env.example antes de ejecutar. La plantilla sirve para conocer nombres, no para conservar credenciales de muestra. Para el backend manual, copiarla a backend/.env y sustituir localmente todo campo sensible. Para Compose, crear un .env en la raíz con las variables que interpola docker-compose.yml. Ambos archivos están ignorados.


```
cp backend/.env.example backend/.env
touch .env
chmod 600 .env backend/.env
# editar ambos archivos localmente; no pegar valores en tickets ni commits
git check-ignore .env backend/.env
git status --short
```

Variable	Dónde	Propósito	Sensible
SECRET_KEY	raíz/Compose y backend manual	firma JWT; generar valor local nuevo	sí
DATABASE_URL	backend manual	SQLite por defecto o URL de PostgreSQL local	sí
MFA_ENCRYPTION_KEY	backend manual	estabilidad de secretos TOTP entre reinicios	sí
DEV_ADMIN_EMAIL / DEV_ADMIN_PASSWORD	seed local	cuenta tenant descartable	sí/parcial
DEV_PLATFORM_OWNER_*	seed local	cuenta PLATFORM descartable	sí/parcial
SEED_DEV_DATA / SEED MOCK_DATA	backend	habilitar datos de desarrollo y dataset mock	no
VITE_API_PROXY_TARGET	frontend	backend local; Vite usa localhost:8000 por defecto	no

5.4 Preflight de un clon limpio

```
test -f frontend/package-lock.json
cd backend
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -e '[dev]'
alembic heads
cd ..
```

Dos bloqueos observados en 291f533: El lockfile frontend no está versionado aunque Compose ejecuta `npm ci`, y el historial Git contiene dos heads Alembic. Antes de considerar reproducible el repositorio, versionar `package-lock.json` y una migración merge revisada. Como reparación local temporal, generar ambos artefactos y conservarlos visibles en git status.

```
# reparación local temporal del lockfile
docker run --rm -v "$PWD/frontend:/app" -w /app node:20-alpine npm install --package-lock-only

# desde backend/.venv, unir los dos heads observados
cd backend
alembic merge -m 'merge incident and platform branches' b1f2a3c4d5e6 c9d1e2f3a4b5
alembic heads
cd ..
git status --short
```

5.5 Opción A — Docker Compose

1. Completar el preflight y confirmar que Alembic informa un solo head.

2. Ejecutar `docker compose up --build` desde la raíz.
3. Esperar a que db esté healthy y a que backend complete migraciones y seed.
4. Abrir `http://localhost:5173`; consultar `http://localhost:8000/health` y `/api/docs`.
5. Usar `docker compose ps` y `docker compose logs` para documentar el estado.

```
docker compose up --build
# en otra terminal
docker compose ps
docker compose logs --tail 100 db backend frontend
curl -fsS http://localhost:8000/health
```

5.6 Opción B — backend y frontend manuales

```
# terminal 1: backend con SQLite local por defecto
cd backend
source .venv/bin/activate
alembic upgrade head
SEED_DEV_DATA=true SEED MOCK_DATA=true python -m app.db.seed_dev
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000

# terminal 2: frontend
cd frontend
npm install
npm run dev -- --host 127.0.0.1 --port 5173
```

Vite proxifica `/api` a `http://localhost:8000` cuando `VITE_API_PROXY_TARGET` no está definido. El backend expone `/health` y, en desarrollo, Swagger en `/api/docs`.

5.7 Validación y cierre de la sesión local

Comprobación	Esperado	Finalidad
GET <code>http://localhost:8000/health</code>	200	API activa y environment visible
GET <code>http://localhost:8000/api/docs</code>	200	OpenAPI habilitado en desarrollo
GET <code>http://localhost:5173</code>	200	SPA servida por Vite
<code>docker compose ps</code>	3 servicios	db, backend y frontend activos
<code>alembic current</code>	head actual	esquema migrado
<code>git status --short</code>	solo cambios locales entendidos	sin secretos o artefactos accidentales

Datos locales: `docker compose down` conserva el volumen `postgres_data`. `docker compose down -v` elimina la base y el volumen de `node_modules`: usar `-v` únicamente cuando se quiera reiniciar un entorno descartable y no haya datos que conservar.

Fuentes: *README.md*, *backend/.env.example*, *docker-compose.yml*, *backend/pyproject.toml*, *frontend/package.json*, *frontend/vite.config.ts* y *migraciones Alembic*.

6. Backend FastAPI

El backend es una aplicación FastAPI síncrona sobre SQLAlchemy Session. Los routers se registran por sprint/ciclo en `app/api/v1/router.py` y se montan con el prefijo común `/api/v1`. La validación de payloads y respuestas usa Pydantic; las reglas compartidas se concentran en `core/` y `services/`.

6.1 Entrada y middleware

Componente	Archivo	Responsabilidad
FastAPI	<code>app/main.py</code>	metadatos, docs condicionales y health check
Rate limiter	<code>core/rate_limit.py</code>	slowapi, clave por IP y respuesta 429
CORS	<code>app/main.py</code>	allow-list configurable; wildcard controlado en desarrollo
Security headers	<code>core/middleware.py</code>	nosniff, DENY, referrer, permissions policy, CSP y HSTS según entorno
Router v1	<code>api/v1/router.py</code>	agrega 29 routers funcionales
Sesión DB	<code>db/session.py</code>	engine + SessionLocal por request

Detalle: Las cabeceras defensivas se aplican a respuestas de FastAPI. Durante `npm run dev`, el HTML y JavaScript los sirve Vite y no heredan automáticamente el middleware de la API. `ProtectedRoute` tampoco sustituye las verificaciones de autorización del backend.

6.2 Configuración

Settings carga `.env`, normaliza `ENVIRONMENT` y aplica una política diferenciada. Para el recorrido local se usa `development`: si no se define `MFA_ENCRYPTION_KEY` genera una Fernet por proceso y, si falta `SECRET_KEY`, aplica un marcador explícitamente inseguro. Para sesiones locales estables, definir valores propios fuera de Git. `SHOW_DOCS` controla OpenAPI, Swagger y ReDoc.

Fuente: `backend/app/core/config.py`.

6.3 Patrón de endpoint

6. FastAPI valida ruta, query y body contra el esquema Pydantic.
7. `Depends(get_current_user)` decodifica el Bearer JWT, carga el usuario y comprueba actividad/inactividad.
8. `require_permission` verifica scope TENANT y la acción RBAC; `require_platform_permission` hace lo propio para PLATFORM.
9. `get_current_tenant_id` obtiene el tenant del usuario y rechaza overrides generales.

10. La consulta filtra por tenant_id; los writes asignan tenant_id desde la dependencia, no desde el payload.
11. Las operaciones sensibles crean AuditLog con actor, recurso, detalle e IP cuando aplica.

6.4 Auditoría

AuditLog es una tabla Base, no TenantBase, porque admite tenant_id y user_id nulos. El modelo expone create_log como vía de escritura y deliberadamente no ofrece métodos de actualización/eliminación. La API permite listar por acción, usuario y fechas, y exportar CSV dentro del tenant. Se encontraron 60 invocaciones AuditLog.create_log en app/.

Recomendación: La inmutabilidad es convencional a nivel ORM. Para garantías fuertes, añadir permisos DB append-only, retención/WORM, integridad encadenada o envío a un SIEM externo.

Fuentes: `backend/app/models/audit_log.py` y `backend/app/api/v1/audit.py`.

7. Autenticación, MFA, RBAC y sesiones

Flujo de autenticación y sesión

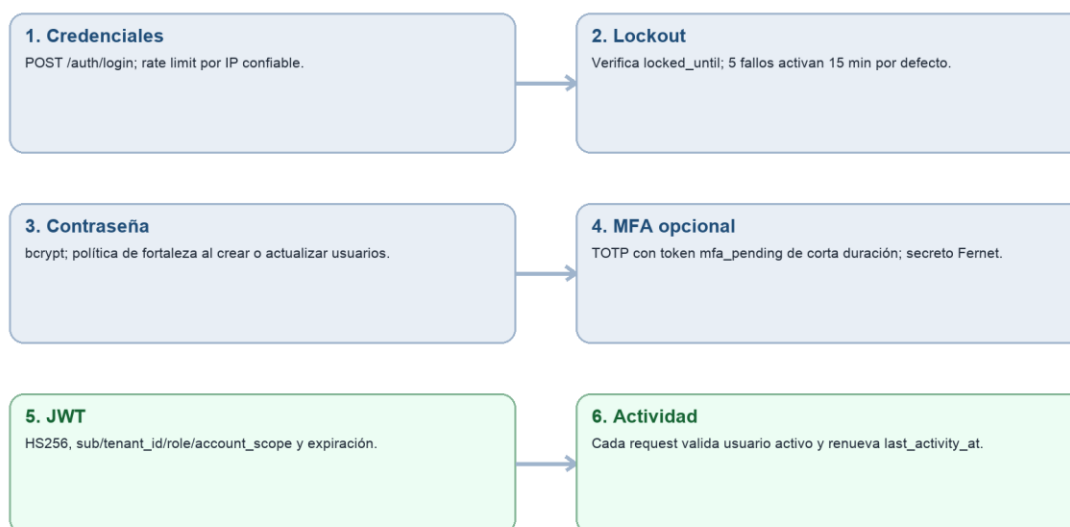


Figura 2. Autenticación desde credenciales hasta actividad autenticada.

7.1 Login y bloqueo

POST /api/v1/auth/login está limitado por AUTH_RATE_LIMIT; la IP considera X-Forwarded-For solo si el proxy inmediato pertenece a TRUSTED_PROXIES.

El usuario se busca por email; las credenciales inválidas responden 401 sin diferenciar usuario/contraseña.

MAX_FAILED_ATTEMPTS=5 y LOCKOUT_MINUTES=15 son los valores por defecto configurables.

Al llegar al umbral se establece locked_until y se reinicia el contador; durante el bloqueo la API responde 423.

Un login correcto limpia failed_attempts/locked_until y actualiza last_activity_at.

7.2 Contraseñas y JWT

Passlib usa bcrypt; la política exige mínimo 8 caracteres, mayúscula, minúscula, número y carácter especial.

La política se valida al crear o actualizar usuarios mediante esquemas Pydantic; el seed local hashea directamente los valores suministrados por entorno.

El token de acceso usa HS256 e incluye sub, tenant_id, role y account_scope; expira en 30 minutos por defecto.

El backend vuelve a cargar al usuario por sub y no confía solo en role/tenant_id del JWT para autorizar.

Cada request autenticado actualiza last_activity_at y aplica expiración por inactividad con el mismo límite del token.

7.3 MFA

mfa-setup genera un secreto TOTP y URI; guarda el secreto cifrado con Fernet, pero no activa MFA hasta confirmar un código.

mfa-confirm valida TOTP y habilita mfa_enabled.

Si MFA está activo, login devuelve mfa_pending de cinco minutos por defecto; mfa-verify emite el JWT completo.

verify_totp acepta una ventana de ± 1 intervalo; los secretos plaintext heredados se detectan y leen por compatibilidad.

Cambiar MFA_ENCRYPTION_KEY sin plan de rotación vuelve indescifrables los secretos existentes.

7.4 Modelo de autorización

Rol/cuenta	Scope	Perfil
SUPER_ADMIN	tenant	administración completa del tenant; no equivale a cuenta PLATFORM
DPO	tenant	gestión amplia de cumplimiento y usuarios; backup solo lectura
ADMIN	tenant	operación amplia con restricciones en documentos/auditoría/backups

Rol/cuenta	Scope	Perfil
DEPT_HEAD	tenant	operación/lectura acotada a módulos y, en algunos recursos, a departamento
AUDITOR	tenant	lectura general, exportes de auditoría y gestión de planes de auditoría
Custom role	tenant	acciones c/r/u/d/export persistidas por módulo
PLATFORM	platform	permisos explícitos tenants:read y/o tenants:provision

7.5 Matriz RBAC del sistema

Módulo	SUPER	DPO	ADMIN	DEPT	AUDIT
action_plans	CRUD	CRUD	CRU	R	R
alerts	CRUD	CRUD	CRUD	RU	R
arco	CRUD	CRUD	CRU	CR	R
audit	RX	RX	R		RX
audit_plans	CRUD	CRUD	R	R	CRU
backups	CRUD	R	CRU		
catalogs	CRUD	CRUD	R		
consents	CRUD	CRUD	CRU	R	R
data_inventory	R	R	R	R	R
departments	CRUD	CRUD	CRU	R	R
dpias	CRUD	CRUD	CRU	CR	R
incidents	CRUD	CRUD	CRU	CR	R
information_assets	CRUD	CRUD	CRU	CRU	R
legal_documents	CRUD	CRUD	R	R	R
permissions	CRUD	CRUD	CRU	R	R
portability	CRUDX	CRUDX	CR	R	R
remediations	CRUD	CRUD	CRU	R	R
reports	R	R	R	R	R
retention	CRUD	CRUD	CRU	R	R
risk_assessments	CRUD	CRUD	CRU	CR	R
ropa	R	R	R	R	R
sectors	RU	RU	RU	R	R
tenants	CRUD				
training	CRUD	CRUD	CRUD	RU	

Módulo	SUPER	DPO	ADMIN	DEPT	AUDIT
treatment_activities	CRUD	CRUD	CRU	CRU	R
users	CRUD	CRUD	CRUD	R	R

Leyenda: C crear · R leer · U actualizar · D eliminar · X exportar. Fuente: `backend/app/core/permissions.py`.

7.6 Consideraciones frontend

El frontend almacena `access_token`, `role`, `tenant_id`, `account_scope` y permisos de plataforma en `localStorage`; el token MFA temporal usa `sessionStorage`. Axios agrega `Authorization` y, ante 401 fuera de login/MFA, limpia el estado y redirige a `/login`.

Riesgo: Un JWT en `localStorage` es accesible para JavaScript y aumenta el impacto de XSS. Evaluar cookies `HttpOnly/Secure/SameSite`, refresh tokens rotativos y CSP estricta en la capa que sirve la SPA.

Fuentes: `backend/app/api/v1/auth.py`, `core/security.py`, `core/mfa_crypto.py`, `api/deps.py`; `frontend/src/hooks/useAuth.ts` y `api/client.ts`.

8. Multi-tenancy, modelos y migraciones

8.1 Estrategia multi-tenant

`TenantBase` hereda `Base` y `TimestampMixin` y añade `tenant_id` indexado. La mayoría de las tablas de negocio heredan `TenantBase`; `Tenant`, `AuditLog` y `PlatformPermission` usan `Base` directamente por sus responsabilidades globales. Cada router `tenant-scoped` obtiene el `tenant` desde el usuario autenticado y filtra explícitamente.

Aislamiento: filas compartidas en una base, diferenciadas por `tenant_id`.

No hay Row-Level Security de PostgreSQL ni esquemas separados; el aislamiento depende del código y pruebas.

`get_current_tenant_id` rechaza query `tenant_id` para endpoints normales; las operaciones de plataforma usan dependencias específicas.

`DEPT_HEAD` recibe filtrado adicional en ciertos recursos, por ejemplo incidentes; no es una regla ORM global.

`User.email` es `unique` global, por lo que dos tenants no pueden reutilizar el mismo correo.

Recomendación: Agregar pruebas sistemáticas de referencias cruzadas entre tenants y considerar RLS. Por ejemplo, `departments` acepta `head_user_id` sin comprobar en ese router que el usuario pertenezca al mismo tenant.

8.2 Inventario de modelos

Modelo / tabla	Campos	Relaciones ORM	Fuente
ActionPlanTemplate action_plan_templates	tenant_id, created_at, updated_at, id, name, description, applies_to_level, default_tasks, is_active	—	backend/app/models/action_plan.py:11
ActionPlan action_plans	tenant_id, created_at, updated_at, id, risk_assessment_id, template_id, title, description, status, tasks, target_date, created_by_id, auto_generated	—	backend/app/models/action_plan.py:25
Alert alerts	tenant_id, created_at, updated_at, id, alert_type, title, message, severity, resource_type, resource_id, recipient_id, is_read, read_at	—	backend/app/models/alert.py:11
ARCOResult arco_requests	tenant_id, created_at, updated_at, id, ticket_number, request_type, requester_name, requester_email, requester_id_number, description, status, received_date, deadline_date, assigned_dpo_id, response_text, responded_at, rejection_reason	—	backend/app/models/arco_request.py:11
AuditLog audit_logs	id, tenant_id, user_id, action, resource, detail, ip_address, created_at	user	backend/app/models/audit_log.py:9
AuditPlan audit_plans	tenant_id, created_at, updated_at, id, name, scope, period_start, period_end, auditor_id, status, notes	—	backend/app/models/audit_plan.py:20
AuditFinding audit_findings	tenant_id, created_at, updated_at, id, audit_plan_id, title, description, evidence, severity, status, remediation_notes, remediation_date, created_by_id	—	backend/app/models/audit_plan.py:36
BackupRecord backup_records	id, tenant_id, filename, checksum_sha256, size_bytes, status, notes, created_at, verified_at	—	backend/app/models/backup.py:17

8.2 Inventario de modelos — continuación (2/5)

Modelo / tabla	Campos	Relaciones ORM	Fuente
CatalogEntry catalog_entries	tenant_id, created_at, updated_at, id, type, code, label, description, is_active, sensitivity, criticality, version, updated_by_id	—	backend/app/models/catalog.py:13
ConsentRecord consent_records	tenant_id, created_at, updated_at, id, treatment_activity_id, data_subject_token, legal_basis, purpose, is_sensitive, granted_at, is_revoked, revoked_at, revocation_reason, created_by_id	—	backend/app/models/consent.py:13
CookieBanner cookie_banners	tenant_id, created_at, updated_at, id, version, content, effective_date, is_active, created_by_id	—	backend/app/models/consent.py:43
CookieConsent cookie_consents	tenant_id, created_at, updated_at, id, banner_id, data_subject_token, accepted_at, revoked_at, is_revoked	—	backend/app/models/consent.py:58
Department departments	tenant_id, created_at, updated_at, id, name, head_user_id	head_user, users	backend/app/models/department.py:7
DPIAssessment dpi_assessments	tenant_id, created_at, updated_at, id, treatment_activity_id, step1_description, step2_risk_analysis, step3_mitigations, status, signed_at, signed_by_id, pdf_bytes, version	—	backend/app/models/dpia.py:11
Incident incidents	tenant_id, created_at, updated_at, id, title, description, incident_type, severity, status, vulnerability_types, regulatory_notification_required, regulatory_notified_at, reporter_id, assigned_to_id, resolved_at, affected_data_types, department_id, delegate_name, delegate_email, delegate_phone, controller_name, controller_email, controller_phone, closure_summary, closure_report_pdf, closed_at	—	backend/app/models/incident.py:17
InformationAsset information_assets	tenant_id, created_at, updated_at, id, name, description, asset_type_code, format_code, storage_medium_code, classification_level_code, treatment_activity_id, department_id, owner_id	treatment_activity, retention_records	backend/app/models/information_asset.py:9

8.2 Inventario de modelos — continuación (3/5)

Modelo / tabla	Campos	Relaciones ORM	Fuente
LegalDocument legal_documents	tenant_id, created_at, updated_at, id, doc_type, title, version, effective_date, parameters, content, is_current, created_by_id	—	backend/app/models/legal_document.py:21
PlatformPermission platform_permissions	created_at, updated_at, id, user_id, permission	user	backend/app/models/platform_access.py:7
PortabilityRequest portability_requests	tenant_id, created_at, updated_at, id, subject_name, subject_email, request_date, status, response_data, completed_at, notes	—	backend/app/models/portability.py:9
Remediation remediations	tenant_id, created_at, updated_at, id, risk_assessment_id, title, description, responsible_id, due_date, completed_date, status, risk_score_before, risk_score_after, risk_level_before, risk_level_after, created_by_id	—	backend/app/models/remediation.py:13
RetentionPolicy retention_policies	tenant_id, created_at, updated_at, id, name, data_category, retention_days, action_on_expiry, legal_basis, is_active	records	backend/app/models/retention.py:11
RetentionRecord retention_records	tenant_id, created_at, updated_at, id, information_asset_id, policy_id, expiry_date, status, legal_hold, hold_justification, review_decision, decision_rationale, reviewed_by_id	information_asset, policy	backend/app/models/retention.py:28
RetentionExecutionLog retention_execution_logs	tenant_id, created_at, updated_at, id, policy_id, executed_by_id, records_processed, records_exceptions, status, log_details, run_type	—	backend/app/models/retention.py:60
RiskAssessment risk_assessments	tenant_id, created_at, updated_at, id, treatment_activity_id, analyst_id, responses, probability, impact, risk_score, risk_level, status, notes	treatment_activity	backend/app/models/risk_assessment.py:22

8.2 Inventario de modelos — continuación (4/5)

Modelo / tabla	Campos	Relaciones ORM	Fuente
Role roles	tenant_id, created_at, updated_at, id, name, description	permissions	backend/app/models/role.py:14
RolePermission role_permissions	tenant_id, created_at, updated_at, id, role_id, module, actions	role	backend/app/models/role.py:32
Tenant tenants	created_at, updated_at, id, name, ruc, country, sector, is_active, address, website, dpo_name, dpo_email, dpo_phone	—	backend/app/models/tenant.py:7
TrainingProgram training_programs	tenant_id, created_at, updated_at, id, title, description, is_active	modules, enrollments	backend/app/models/training.py:9
TrainingModule training_modules	tenant_id, created_at, updated_at, id, program_id, title, description, order	program, materials	backend/app/models/training.py:26
TrainingMaterial training_materials	tenant_id, created_at, updated_at, id, module_id, title, content_type, url, content	module	backend/app/models/training.py:44
Enrollment enrollments	tenant_id, created_at, updated_at, id, user_id, program_id, enrolled_at, completed_at, progress_pct	user, program	backend/app/models/training.py:60
TreatmentActivity treatment_activities	tenant_id, created_at, updated_at, id, name, purpose, legal_basis, personal_data_types, data_subjects, retention_period_days, is_cross_border, destination_countries, processor_name, processor_country, department_id, owner_id, status	information_assets, risk_assessments	backend/app/models/treatment_activity.py:9

8.2 Inventario de modelos — continuación (5/5)

Modelo / tabla	Campos	Relaciones ORM	Fuente
User users	tenant_id, created_at, updated_at, id, email, hashed_password, full_name, role, account_scope, department_id, is_active, failed_attempts, locked_until, mfa_secret, mfa_enabled, last_activity_at	department, headed_department, audit_logs, enrollments, platform_permissions	backend/app/models/user.py:9

Inventario AST: 33 clases con `__tablename__`. Los tipos y restricciones exactos se consultan en cada archivo indicado.

8.3 Relaciones funcionales principales

Relación	Uso
Tenant → datos	tenant_id lógico en usuarios, departamentos, tratamientos, riesgos, derechos, operaciones y documentos
Department ↔ User	head_user_id y department_id forman relaciones bidireccionales
TreatmentActivity → InformationAsset	los activos pueden enlazarse a una actividad y a catálogos de tipo/formato/medio/clasificación
TreatmentActivity → Risk/DPIA/Consent	los procesos de cumplimiento referencian la actividad de tratamiento
RiskAssessment → ActionPlan/Remediation	planes y remediaciones conservan vínculo y snapshots de score
AuditPlan → AuditFinding	hallazgos por plan, tenant y severidad
TrainingProgram → Module → Material	jerarquía de capacitación; Enrollment enlaza programa y usuario
Role → RolePermission	acciones por módulo con unique(role_id,module)
User → PlatformPermission	permisos globales para cuentas account_scope=PLATFORM

8.4 Esquemas Pydantic

Dominio	Clases	Archivo
action_plan	5	backend/app/schemas/action_plan.py
alert	3	backend/app/schemas/alert.py
arco	3	backend/app/schemas/arco.py
audit_plan	6	backend/app/schemas/audit_plan.py
auth	7	backend/app/schemas/auth.py
backup	3	backend/app/schemas/backup.py
catalog	3	backend/app/schemas/catalog.py
consent	8	backend/app/schemas/consent.py

Dominio	Clases	Archivo
department	3	backend/app/schemas/department.py
dpia	3	backend/app/schemas/dpia.py
incident	4	backend/app/schemas/incident.py
information_asset	3	backend/app/schemas/information_asset.py
legal_document	3	backend/app/schemas/legal_document.py
portability	5	backend/app/schemas/portability.py
remediation	3	backend/app/schemas/remediation.py
retention	8	backend/app/schemas/retention.py
risk_assessment	4	backend/app/schemas/risk_assessment.py
role	5	backend/app/schemas/role.py
tenant	4	backend/app/schemas/tenant.py
training	12	backend/app/schemas/training.py
treatment_activity	3	backend/app/schemas/treatment_activity.py
user	3	backend/app/schemas/user.py

Total verificado: 101 clases de esquema. Algunos reportes definen esquemas locales en su router.

8.5 Alembic

Revisión	Padre	Contenido	Estado Git
9ea2fa322267	base	baseline sprints 1–5	versionado
b1f2b4650c86	9ea2fa322267	roles personalizados	versionado
c9d1e2f3a4b5	b1f2b4650c86	account_scope + platform_permissions	versionado · head
b1f2a3c4d5e6	9ea2fa322267	campos SSPDP de incidentes	versionado · head

Estado del clon 291f533: alembic heads devuelve b1f2a3c4d5e6 y c9d1e2f3a4b5. Por ello, alembic upgrade head es ambiguo hasta que una migración merge revisada se incorpore al historial Git. La migración de incidentes omite DROP DEFAULT en SQLite y lo ejecuta en PostgreSQL.

```
cd backend
alembic heads
alembic history --verbose
# reparación local temporal; revisar y versionar el archivo generado
alembic merge -m 'merge incident and platform branches' b1f2a3c4d5e6 c9d1e2f3a4b5
alembic heads
alembic upgrade head
```

Fuentes: `backend/alembic/env.py`, `backend/alembic/versions/` y ejecución de `alembic heads/history`.

9. Inventario completo de endpoints

La aplicación registra 156 operaciones bajo `/api/v1`, distribuidas en 29 routers, más `GET /health` fuera de la versión de API. El inventario siguiente se extrajo de decoradores `@router` mediante AST; por eso representa el código actual y no una lista manual del README.

Router	Ops.	Responsabilidad
Actividades de tratamiento	9	wizard y ciclo de vida de tratamientos
Activos de información	5	inventario técnico y clasificación
Administración de tenants	3	lectura y provisión desde cuentas PLATFORM
Alertas	6	bandeja interna y escaneo de plazos ARCO/auditoría
Autenticación y MFA	4	login, MFA TOTP, bloqueo y emisión JWT
Bitácora de auditoría	2	consulta y exportación CSV de eventos append-only
Capacitación	14	programas, módulos, materiales e inscripciones
Catálogos maestros	5	catálogos configurables y control de referencias
Consentimientos y cookies	10	consentimientos, revocación, banners y cookies
Departamentos	5	estructura organizativa y responsables
Documentos legales	5	plantillas, versiones y PDF
Evaluaciones DPIA	6	flujo DPIA, firma y PDF almacenado
Evaluaciones de riesgo	6	cuestionario, score y dashboard
Importación y exportación	3	carga masiva y exportación de snapshots
Incidentes	7	registro, notificación, cierre y reporte PDF
Inventario de datos	1	progreso agregado del inventario de cumplimiento
Perfil de compañía	2	datos institucionales y contacto del DPO
Planes de acción	7	plantillas, creación manual y generación automática basada en riesgos
Planes de auditoría	8	planes, hallazgos y reporte PDF
Portabilidad	5	solicitudes y exportación de datos
ROPA	2	registro consolidado en JSON y PDF
Remediaciones	4	acciones correctivas enlazadas a riesgos
Reportes y KPI	5	resumen, KPI, tendencias y exportes PDF/CSV
Respallos	4	registros, snapshots SQLite y verificación de checksum
Retención	9	políticas, registros, ejecución y revisiones
Roles personalizados	5	roles de tenant y permisos configurables
Sectores económicos	3	sugerencias sectoriales y perfil empresarial
Solicitudes ARCO	6	registro de derechos, asignación y semáforo SLA
Usuarios	5	CRUD, roles, estado y departamento

9.1 Actividades de tratamiento

Propósito: wizard y ciclo de vida de tratamientos. Operaciones: 9.

Método	Ruta	Operación	Fuente
GET	/api/v1/treatment-activities	List activities.	backend/app/api/v1/treatment_activities.py:179
POST	/api/v1/treatment-activities	Create activity.	backend/app/api/v1/treatment_activities.py:199
POST	/api/v1/treatment-activities/wizard/start	Step 0: Create a DRAFT treatment activity with minimal info.	backend/app/api/v1/treatment_activities.py:57
POST	/api/v1/treatment-activities/wizard/{activity_id}/finalize	Step 4: Validate and activate the treatment activity.	backend/app/api/v1/treatment_activities.py:137
PATCH	/api/v1/treatment-activities/wizard/{activity_id}/legal-basis	Step 2: Set legal basis, personal data types and subjects.	backend/app/api/v1/treatment_activities.py:86
PATCH	/api/v1/treatment-activities/wizard/{activity_id}/transfers	Step 3: Set cross-border transfer details.	backend/app/api/v1/treatment_activities.py:112
DELETE	/api/v1/treatment-activities/{activity_id}	Delete activity.	backend/app/api/v1/treatment_activities.py:297
GET	/api/v1/treatment-activities/{activity_id}	Return activity.	backend/app/api/v1/treatment_activities.py:242

9.1 Actividades de tratamiento — continuación (2/2)

Método	Ruta	Operación	Fuente
PATCH	/api/v1/treatment-activities/{activity_id}	Update activity.	backend/app/api/v1/treatment_activities.py:263

9.2 Activos de información

Propósito: inventario técnico y clasificación. Operaciones: 5.

Método	Ruta	Operación	Fuente
GET	/api/v1/information-assets	List assets.	backend/app/api/v1/information_assets.py:29
POST	/api/v1/information-assets	Create asset.	backend/app/api/v1/information_assets.py:49
DELETE	/api/v1/information-assets/{asset_id}	Delete asset.	backend/app/api/v1/information_assets.py:141
GET	/api/v1/information-assets/{asset_id}	Return asset.	backend/app/api/v1/information_assets.py:87
PATCH	/api/v1/information-assets/{asset_id}	Update asset.	backend/app/api/v1/information_assets.py:107

9.3 Administración de tenants

Propósito: lectura y provisión desde cuentas PLATFORM. Operaciones: 3.

Método	Ruta	Operación	Fuente
GET	/api/v1/tenants	List tenants.	backend/app/api/v1/tenants.py:23
POST	/api/v1/tenants/provision	Platform account provisions a new tenant + its first DPO/ADMIN account.	backend/app/api/v1/tenants.py:45
GET	/api/v1/tenants/{tenant_id}	Return tenant.	backend/app/api/v1/tenants.py:32

9.4 Alertas

Propósito: bandeja interna y escaneo de plazos ARCO/auditoría. Operaciones: 6.

Método	Ruta	Operación	Fuente
GET	/api/v1/alerts	List alerts. DPO/Admin see all tenant alerts; other roles see own + broadcasts.	backend/app/api/v1/alerts.py:64
POST	/api/v1/alerts	Create an internal alert (DPO/Admin only).	backend/app/api/v1/alerts.py:86
POST	/api/v1/alerts/scan	Scan open legal deadlines (ARCO SLA, audit plans) and raise missing alerts.	backend/app/api/v1/alerts.py:20
GET	/api/v1/alerts/unread-count	Count of unread alerts for the current user (personal + broadcasts).	backend/app/api/v1/alerts.py:45
DELETE	/api/v1/alerts/{alert_id}	Delete an alert. Only ADMIN/DPO or the owning user may delete.	backend/app/api/v1/alerts.py:156
PATCH	/api/v1/alerts/{alert_id}/read	Mark an alert as read (recipient or DPO/Admin).	backend/app/api/v1/alerts.py:121

9.5 Autenticación y MFA

Propósito: login, MFA TOTP, bloqueo y emisión JWT. Operaciones: 4.

Método	Ruta	Operación	Fuente
POST	/api/v1/auth/login	Handle login.	backend/app/api/v1/auth.py:46
POST	/api/v1/auth/mfa-confirm	Confirm MFA setup by verifying a TOTP code — enables MFA.	backend/app/api/v1/auth.py:233
POST	/api/v1/auth/mfa-setup	Generate a new TOTP secret for the current user.	backend/app/api/v1/auth.py:205
POST	/api/v1/auth/mfa-verify	Handle mfa verify.	backend/app/api/v1/auth.py:139

9.6 Bitácora de auditoría

Propósito: consulta y exportación CSV de eventos append-only. Operaciones: 2.

Método	Ruta	Operación	Fuente
GET	/api/v1/audit	List audit logs.	backend/app/api/v1/audit.py:18
GET	/api/v1/audit/export	Handle export audit csv.	backend/app/api/v1/audit.py:43

9.7 Capacitación

Propósito: programas, módulos, materiales e inscripciones. Operaciones: 14.

Método	Ruta	Operación	Fuente
GET	/api/v1/training/enrollments	List enrollments.	backend/app/api/v1/training.py:276
POST	/api/v1/training/enrollments	Create enrollment.	backend/app/api/v1/training.py:293
PUT	/api/v1/training/enrollments/{enrollment_id}	Update enrollment.	backend/app/api/v1/training.py:314
PUT	/api/v1/training/materials/{material_id}	Update material.	backend/app/api/v1/training.py:250
PUT	/api/v1/training/modules/{module_id}	Update module.	backend/app/api/v1/training.py:175
GET	/api/v1/training/modules/{module_id}/materials	List materials.	backend/app/api/v1/training.py:201
POST	/api/v1/training/modules/{module_id}/materials	Create material.	backend/app/api/v1/training.py:220
GET	/api/v1/training/programs	List programs.	backend/app/api/v1/training.py:32

9.7 Capacitación — continuación (2/2)

Método	Ruta	Operación	Fuente
POST	/api/v1/training/programs	Create program.	backend/app/api/v1/training.py:50
DELETE	/api/v1/training/programs/{program_id}	Delete program.	backend/app/api/v1/training.py:106
GET	/api/v1/training/programs/{program_id}	Return program.	backend/app/api/v1/training.py:65
PUT	/api/v1/training/programs/{program_id}	Update program.	backend/app/api/v1/training.py:83
GET	/api/v1/training/programs/{program_id}/modules	List modules.	backend/app/api/v1/training.py:128
POST	/api/v1/training/programs/{program_id}/modules	Create module.	backend/app/api/v1/training.py:146

9.8 Catálogos maestros

Propósito: catálogos configurables y control de referencias. Operaciones: 5.

Método	Ruta	Operación	Fuente
GET	/api/v1/catalogs	List catalogs.	backend/app/api/v1/catalogs.py:110
POST	/api/v1/catalogs/bulk-load	POST /catalogs/bulk-load — accepts list of entries [{type, code, label, description}].	backend/app/api/v1/catalogs.py:151
GET	/api/v1/catalogs/{catalog_type}	List catalogs by type.	backend/app/api/v1/catalogs.py:130
DELETE	/api/v1/catalogs/{entry_id}	Delete catalog entry.	backend/app/api/v1/catalogs.py:222
PATCH	/api/v1/catalogs/{entry_id}	US-RF20-1: Edit catalog entry with version increment.	backend/app/api/v1/catalogs.py:180

9.9 Consentimientos y cookies

Propósito: consentimientos, revocación, banners y cookies. Operaciones: 10.

Método	Ruta	Operación	Fuente
GET	/api/v1/consents	List consent records with optional filters.	backend/app/api/v1/consents.py:100
POST	/api/v1/consents	Register a new explicit consent record.	backend/app/api/v1/consents.py:69
GET	/api/v1/consents/stats	Consent registration statistics per tenant (US-RF06-1).	backend/app/api/v1/consents.py:33
GET	/api/v1/consents/{record_id}	Retrieve a consent record by ID.	backend/app/api/v1/consents.py:119
POST	/api/v1/consents/{record_id}/revoke	Explicitly and immutably revoke a consent. Once revoked, cannot be un-revoked.	backend/app/api/v1/consents.py:135
GET	/api/v1/cookie-banners	List cookie banners, optionally filtering to active-only.	backend/app/api/v1/consents.py:204
POST	/api/v1/cookie-banners	Create a new versioned cookie consent banner.	backend/app/api/v1/consents.py:175
PATCH	/api/v1/cookie-banners/{banner_id}	Update a cookie banner's content or active status.	backend/app/api/v1/consents.py:218

9.9 Consentimientos y cookies — continuación (2/2)

Método	Ruta	Operación	Fuente
POST	/api/v1/cookie-consents	Record a data subject's acceptance of a cookie banner version.	backend/app/api/v1/consents.py:243
POST	/api/v1/cookie-consents/{consent_id}/revoke	Revoke a previously recorded cookie consent.	backend/app/api/v1/consents.py:267

9.10 Departamentos

Propósito: estructura organizativa y responsables. Operaciones: 5.

Método	Ruta	Operación	Fuente
GET	/api/v1/departments	List departments.	backend/app/api/v1/departments.py:16
POST	/api/v1/departments	Create department.	backend/app/api/v1/departments.py:34
DELETE	/api/v1/departments/{dept_id}	Delete department.	backend/app/api/v1/departments.py:98
GET	/api/v1/departments/{dept_id}	Return department.	backend/app/api/v1/departments.py:57
PUT	/api/v1/departments/{dept_id}	Update department.	backend/app/api/v1/departments.py:75

9.11 Documentos legales

Propósito: plantillas, versiones y PDF. Operaciones: 5.

Método	Ruta	Operación	Fuente
GET	/api/v1/legal-documents	List legal documents with optional type and current-version filters.	backend/app/api/v1/legal_documents.py:95
POST	/api/v1/legal-documents	Create a new versioned legal document from a template type.	backend/app/api/v1/legal_documents.py:51
GET	/api/v1/legal-documents/template-types	List all supported document template types (US-RF14-1 + US-RF33-1).	backend/app/api/v1/legal_documents.py:34
GET	/api/v1/legal-documents/{doc_id}	Retrieve a legal document by ID.	backend/app/api/v1/legal_documents.py:114
GET	/api/v1/legal-documents/{doc_id}/pdf	Generate and download a PDF for the legal document (US-RF14-1).	backend/app/api/v1/legal_documents.py:130

9.12 Evaluaciones DPIA

Propósito: flujo DPIA, firma y PDF almacenado. Operaciones: 6.

Método	Ruta	Operación	Fuente
GET	/api/v1/dpias	List all DPIAs for the current tenant.	backend/app/api/v1/dpia.py:99
POST	/api/v1/dpias	Create a new DPIA for a treatment activity.	backend/app/api/v1/dpia.py:70
GET	/api/v1/dpias/{dpia_id}	Retrieve a single DPIA by ID.	backend/app/api/v1/dpia.py:109
PATCH	/api/v1/dpias/{dpia_id}	Update DPIA steps or status.	backend/app/api/v1/dpia.py:125
GET	/api/v1/dpias/{dpia_id}/pdf	Return the stored DPIA PDF bytes.	backend/app/api/v1/dpia.py:180
POST	/api/v1/dpias/{dpia_id}/sign	DPO or SUPER_ADMIN signs the DPIA, generates and stores the PDF.	backend/app/api/v1/dpia.py:146

9.13 Evaluaciones de riesgo

Propósito: cuestionario, score y dashboard. Operaciones: 6.

Método	Ruta	Operación	Fuente
GET	/api/v1/risk-assessments	List assessments.	backend/app/api/v1/risk_assessments.py:130
POST	/api/v1/risk-assessments	Create assessment.	backend/app/api/v1/risk_assessments.py:75
GET	/api/v1/risk-assessments/dashboard	US-RF10-1: Risk score summary with GREEN/YELLOW/RED counts.	backend/app/api/v1/risk_assessments.py:38
GET	/api/v1/risk-assessments/questionnaire	Return the standard risk questionnaire structure.	backend/app/api/v1/risk_assessments.py:30
GET	/api/v1/risk-assessments/{assessment_id}	Return assessment.	backend/app/api/v1/risk_assessments.py:147
PATCH	/api/v1/risk-assessments/{assessment_id}	Update assessment.	backend/app/api/v1/risk_assessments.py:165

9.14 Importación y exportación

Propósito: carga masiva y exportación de snapshots. Operaciones: 3.

Método	Ruta	Operación	Fuente
GET	/api/v1/export/compliance-report	Return a full JSON compliance snapshot for the tenant.	backend/app/api/v1/import_export.py:156
GET	/api/v1/export/treatment-activities	Export all treatment activities for the tenant as a CSV file (streamed).	backend/app/api/v1/import_export.py:104
POST	/api/v1/import/treatment-activities	Bulk-import treatment activities from a JSON payload.	backend/app/api/v1/import_export.py:47

9.15 Incidentes

Propósito: registro, notificación, cierre y reporte PDF. Operaciones: 7.

Método	Ruta	Operación	Fuente
GET	/api/v1/incidents	List incidents.	backend/app/api/v1/incidents.py:103
POST	/api/v1/incidents	Create incident.	backend/app/api/v1/incidents.py:124
GET	/api/v1/incidents/{incident_id}	Return incident.	backend/app/api/v1/incidents.py:194
PATCH	/api/v1/incidents/{incident_id}	Update incident.	backend/app/api/v1/incidents.py:214
POST	/api/v1/incidents/{incident_id}/close	Close an incident, generating and storing a PDF closure report (DAT-52).	backend/app/api/v1/incidents.py:288
POST	/api/v1/incidents/{incident_id}/notify	Mark that regulatory authority has been notified (US-RF08-1).	backend/app/api/v1/incidents.py:258
GET	/api/v1/incidents/{incident_id}/report.pdf	Download the stored PDF closure report for an incident (DAT-52).	backend/app/api/v1/incidents.py:351

9.16 Inventario de datos

Propósito: progreso agregado del inventario de cumplimiento. Operaciones: 1.

Método	Ruta	Operación	Fuente
GET	/api/v1/data-inventory/progress	US-RF28-1: Returns automatic progress and reports for data inventory.	backend/app/api/v1/data_inventory.py:41

9.17 Perfil de compañía

Propósito: datos institucionales y contacto del DPO. Operaciones: 2.

Método	Ruta	Operación	Fuente
GET	/api/v1/company-profile	Return the current tenant's company profile including DPO info.	backend/app/api/v1/company_profile.py:18
PUT	/api/v1/company-profile	Update company profile. Only DPO or SUPER_ADMIN may update.	backend/app/api/v1/company_profile.py:31

9.18 Planes de acción

Propósito: plantillas, creación manual y generación automática basada en riesgos. Operaciones: 7.

Método	Ruta	Operación	Fuente
GET	/api/v1/action-plans	List action plans with optional status and risk assessment filters.	backend/app/api/v1/action_plans.py:158
POST	/api/v1/action-plans	Create a new action plan.	backend/app/api/v1/action_plans.py:134
POST	/api/v1/action-plans/auto-generate	Auto-generate action plans for all HIGH/MEDIUM risk assessments without a plan.	backend/app/api/v1/action_plans.py:68
GET	/api/v1/action-plans/templates	List all active action plan templates.	backend/app/api/v1/action_plans.py:50
POST	/api/v1/action-plans/templates	Create a reusable action plan template.	backend/app/api/v1/action_plans.py:29
GET	/api/v1/action-plans/{plan_id}	Retrieve an action plan by ID.	backend/app/api/v1/action_plans.py:175
PATCH	/api/v1/action-plans/{plan_id}	Update an action plan's fields.	backend/app/api/v1/action_plans.py:191

9.19 Planes de auditoría

Propósito: planes, hallazgos y reporte PDF. Operaciones: 8.

Método	Ruta	Operación	Fuente
GET	/api/v1/audit-plans	List audit plans for the current tenant, optionally filtered by status.	backend/app/api/v1/audit_plans.py:55
POST	/api/v1/audit-plans	Create a new audit plan for the current tenant.	backend/app/api/v1/audit_plans.py:28
POST	/api/v1/audit-plans/findings	Register a new finding under an audit plan.	backend/app/api/v1/audit_plans.py:109
PATCH	/api/v1/audit-plans/findings/{finding_id}	Update an existing audit finding.	backend/app/api/v1/audit_plans.py:170
GET	/api/v1/audit-plans/{plan_id}	Retrieve a single audit plan by id within the current tenant.	backend/app/api/v1/audit_plans.py:71
PATCH	/api/v1/audit-plans/{plan_id}	Update an existing audit plan.	backend/app/api/v1/audit_plans.py:86
GET	/api/v1/audit-plans/{plan_id}/findings	List findings for an audit plan, optionally filtered by severity/status.	backend/app/api/v1/audit_plans.py:145
GET	/api/v1/audit-plans/{plan_id}/report	Generate and download a PDF audit report with all findings (US-RF13-1).	backend/app/api/v1/audit_plans.py:193

9.20 Portabilidad

Propósito: solicitudes y exportación de datos. Operaciones: 5.

Método	Ruta	Operación	Fuente
GET	/api/v1/portability	List portability requests.	backend/app/api/v1/portability.py:25
POST	/api/v1/portability	Create portability request.	backend/app/api/v1/portability.py:41
GET	/api/v1/portability/{request_id}	Return portability request.	backend/app/api/v1/portability.py:71
PUT	/api/v1/portability/{request_id}/complete	Handle complete portability request.	backend/app/api/v1/portability.py:91
GET	/api/v1/portability/{request_id}/export	Handle export portability request.	backend/app/api/v1/portability.py:139

9.21 ROPA

Propósito: registro consolidado en JSON y PDF. Operaciones: 2.

Método	Ruta	Operación	Fuente
GET	/api/v1/ropa	Return the Record of Processing Activities (ROPA) report as JSON (US-RF31-1).	backend/app/api/v1/ropa.py:20
GET	/api/v1/ropa/pdf	Return the ROPA report as a downloadable PDF (US-RF31-1).	backend/app/api/v1/ropa.py:30

9.22 Remediaciones

Propósito: acciones correctivas enlazadas a riesgos. Operaciones: 4.

Método	Ruta	Operación	Fuente
GET	/api/v1/remediations	List remediations with optional risk assessment and status filters.	backend/app/api/v1/remediations.py:60
POST	/api/v1/remediations	Create a remediation action linked to a risk assessment.	backend/app/api/v1/remediations.py:19
GET	/api/v1/remediations/{remediation_id}	Retrieve a remediation by ID.	backend/app/api/v1/remediations.py:79
PATCH	/api/v1/remediations/{remediation_id}	Update a remediation's status, dates, or risk impact scores.	backend/app/api/v1/remediations.py:95

9.23 Reportes y KPI

Propósito: resumen, KPI, tendencias y exportes PDF/CSV. Operaciones: 5.

Método	Ruta	Operación	Fuente
GET	/api/v1/reports/kpis	Key performance indicators for executive dashboard (US-RF16-1).	backend/app/api/v1/reports.py:234
GET	/api/v1/reports/summary	Consolidated compliance summary report (US-RF42-1).	backend/app/api/v1/reports.py:178
GET	/api/v1/reports/summary/csv	Download the consolidated summary report as CSV (US-RF15-1).	backend/app/api/v1/reports.py:590
GET	/api/v1/reports/summary/pdf	Download the consolidated summary report as PDF (US-RF15-1).	backend/app/api/v1/reports.py:492
GET	/api/v1/reports/trends	Monthly trend data for the last N months (default 6) — US-RF16-1.	backend/app/api/v1/reports.py:344

9.24 RespalDOS

Propósito: registros, snapshots SQLite y verificación de checksum. Operaciones: 4.

Método	Ruta	Operación	Fuente
GET	/api/v1/backups	List backup records (filtered to current tenant or all if SUPER_ADMIN).	backend/app/api/v1/backups.py:121
POST	/api/v1/backups/create	Create a database backup snapshot and record its checksum.	backend/app/api/v1/backups.py:34
GET	/api/v1/backups/{backup_id}	Retrieve a single backup record by ID.	backend/app/api/v1/backups.py:136
POST	/api/v1/backups/{backup_id}/verify	Verify backup file integrity by recomputing its SHA-256 checksum.	backend/app/api/v1/backups.py:152

9.25 Retención

Propósito: políticas, registros, ejecución y revisiones. Operaciones: 9.

Método	Ruta	Operación	Fuente
POST	/api/v1/retention/execute	Run retention policies: mark expired ACTIVE records as UNDER_REVIEW.	backend/app/api/v1/retention.py:223
GET	/api/v1/retention/execution-logs	List retention execution logs for this tenant.	backend/app/api/v1/retention.py:284
GET	/api/v1/retention/expired-under-review	Return all records where expiry_date has passed AND legal_hold=True, requiring DPO decision.	backend/app/api/v1/retention.py:181
GET	/api/v1/retention/policies	List policies.	backend/app/api/v1/retention.py:32
POST	/api/v1/retention/policies	Create policy.	backend/app/api/v1/retention.py:46
PATCH	/api/v1/retention/policies/{policy_id}	Update policy.	backend/app/api/v1/retention.py:76
GET	/api/v1/retention/records	List records.	backend/app/api/v1/retention.py:102
POST	/api/v1/retention/records	Create record.	backend/app/api/v1/retention.py:119

9.25 Retención — continuación (2/2)

Método	Ruta	Operación	Fuente
PATCH	/api/v1/retention/records/{record_id}	Update record.	backend/app/api/v1/retention.py:141

9.26 Roles personalizados

Propósito: roles de tenant y permisos configurables. Operaciones: 5.

Método	Ruta	Operación	Fuente
GET	/api/v1/roles	List every role available to the tenant: built-in system roles + custom roles.	backend/app/api/v1/roles.py:48
POST	/api/v1/roles	Create a custom role with configurable, per-module access.	backend/app/api/v1/roles.py:79
DELETE	/api/v1/roles/{role_id}	Delete a custom role. Refused while any user is still assigned to it.	backend/app/api/v1/roles.py:154
PATCH	/api/v1/roles/{role_id}	Update a custom role's description and/or module permissions.	backend/app/api/v1/roles.py:113
GET	/api/v1/roles/{role_name}	Return the permission matrix for a single role (US-RF: role-permission query view).	backend/app/api/v1/roles.py:61

9.27 Sectores económicos

Propósito: sugerencias sectoriales y perfil empresarial. Operaciones: 3.

Método	Ruta	Operación	Fuente
GET	/api/v1/sectors	List all economic sectors with suggestions (US-RF34-1).	backend/app/api/v1/sectors.py:187
PATCH	/api/v1/sectors/company/sector	Update the company's economic sector and get pre-populated suggestions (US-RF34-1).	backend/app/api/v1/sectors.py:211
GET	/api/v1/sectors/{sector_code}	Get auto-suggestions for a specific economic sector (US-RF34-1).	backend/app/api/v1/sectors.py:195

9.28 Solicitudes ARCO

Propósito: registro de derechos, asignación y semáforo SLA. Operaciones: 6.

Método	Ruta	Operación	Fuente
GET	/api/v1/arco-requests	List ARCO requests with optional type and status filters.	backend/app/api/v1/arco.py:108
POST	/api/v1/arco-requests	Create an ARCO rights request and notify DPO.	backend/app/api/v1/arco.py:56
GET	/api/v1/arco-requests/dashboard	Dashboard summary: totals, by type, by status, overdue count.	backend/app/api/v1/arco.py:28
GET	/api/v1/arco-requests/{request_id}	Retrieve a single ARCO request by ID.	backend/app/api/v1/arco.py:125
PATCH	/api/v1/arco-requests/{request_id}	Update status, assignee, or response of an ARCO request.	backend/app/api/v1/arco.py:143
GET	/api/v1/arco-requests/{request_id}/sla-status	SLA stoplight for an ARCO request (US-RF07-1).	backend/app/api/v1/arco.py:184

9.29 Usuarios

Propósito: CRUD, roles, estado y departamento. Operaciones: 5.

Método	Ruta	Operación	Fuente
GET	/api/v1/users	List users.	backend/app/api/v1/users.py:32
POST	/api/v1/users	Create user.	backend/app/api/v1/users.py:45
DELETE	/api/v1/users/{user_id}	Delete user.	backend/app/api/v1/users.py:130
GET	/api/v1/users/{user_id}	Return user.	backend/app/api/v1/users.py:82
PUT	/api/v1/users/{user_id}	Update user.	backend/app/api/v1/users.py:96

9.30 Health check

Método	Ruta	Operación	Fuente
GET	/health	Estado, versión y environment; no requiere JWT.	backend/app/main.py:53

Contrato vivo: En desarrollo, SHOW_DOCS habilita /api/docs y /api/openapi.json. Para integraciones locales, generar clientes desde ese OpenAPI en lugar de duplicar manualmente los contratos.

10. Frontend React/Vite

La SPA usa React Router con definición declarativa de rutas, lazy loading de páginas y un AppShell común. La navegación lateral se deriva de un árbol separado que aplica las mismas restricciones de rol/scope para ocultar opciones, mientras ProtectedRoute impide renderizar una ruta no autorizada.

10.1 Arranque y composición

- main.tsx inicializa i18next con en.json, monta React.StrictMode y App.
- App.tsx crea BrowserRouter y registra PUBLIC_ROUTES/PROTECTED_ROUTES.
- Cada página se carga con React.lazy y Suspense muestra estados de carga.
- ProtectedRoute valida token, rol, account_scope y permisos de plataforma almacenados.
- AppShell compone Sidebar, Topbar, breadcrumbs y el contenido de página.

Fuentes: *frontend/src/main.tsx*, *App.tsx*, *routes/routes.tsx* y *components/ProtectedRoute.tsx*.

10.2 Autenticación y cliente API

Archivo	Responsabilidad
api/client.ts	Axios baseURL=/api/v1; Bearer interceptor; limpia sesión en 401
hooks/useAuth.ts	lee/escribe token, rol, tenant, scope y permisos en localStorage
api/auth.ts	login, verificación y configuración MFA
routes/permissions.ts	helpers de rol, scope y plataforma
ProtectedRoute.tsx	redirección a login o estado Forbidden

10.3 Inventario de rutas

Ruta	Página	Acceso UI	Archivo
/login	Login	pública	frontend/src/pages/Login.tsx

Ruta	Página	Acceso UI	Archivo
/mfa-verify	MFAVerify	pública	frontend/src/pages/MFAVerify.tsx
/mfa-setup	MFASetup	scope=TENANT/PLATFORM	frontend/src/pages/MFASetup.tsx
/platform	PlatformDashboard	scope=PLATFORM; perm=tenants:read	frontend/src/pages/PlatformDashboard.tsx
/dashboard	Dashboard	scope=TENANT	frontend/src/pages/Dashboard.tsx
/alerts	Alerts	scope=TENANT	frontend/src/pages/Alerts.tsx
/reports	Reports	scope=TENANT; roles=SUPER_ADMIN,DPO,ADMIN,DEPT_HEAD,AUDITOR	frontend/src/pages/Reports.tsx
/users	Users	scope=TENANT; roles=SUPER_ADMIN,DPO,ADMIN	frontend/src/pages/UserManagement.tsx
/departments	Departments	scope=TENANT; roles=SUPER_ADMIN,DPO,ADMIN	frontend/src/pages/Departments.tsx
/company-profile	CompanyProfile	scope=TENANT; roles=SUPER_ADMIN,DPO,ADMIN	frontend/src/pages/CompanyProfile.tsx
/catalogs	Catalogs	scope=TENANT; roles=SUPER_ADMIN,DPO,ADMIN	frontend/src/pages/Catalogs.tsx
/sectors	Sectors	scope=TENANT; roles=SUPER_ADMIN,DPO,ADMIN	frontend/src/pages/Sectors.tsx
/tenants	Tenants	scope=PLATFORM; perm=tenants:read	frontend/src/pages/Tenants.tsx
/data-inventory	DataInventory	scope=TENANT; roles=SUPER_ADMIN,DPO,ADMIN,DEPT_HEAD	frontend/src/pages/DataInventory.tsx
/treatment-activities	TreatmentActivities	scope=TENANT; roles=SUPER_ADMIN,DPO,ADMIN,DEPT_HEAD	frontend/src/pages/TreatmentActivities.tsx
/information-assets	InformationAssets	scope=TENANT; roles=SUPER_ADMIN,DPO,ADMIN,DEPT_HEAD	frontend/src/pages/InformationAssets.tsx
/retention	Retention	scope=TENANT; roles=SUPER_ADMIN,DPO,ADMIN	frontend/src/pages/Retention.tsx
/import-export	ImportExport	scope=TENANT; roles=SUPER_ADMIN,DPO,ADMIN	frontend/src/pages/ImportExport.tsx
/risk-assessments	RiskAssessments	scope=TENANT; roles=SUPER_ADMIN,DPO,ADMIN,DEPT_HEAD	frontend/src/pages/RiskAssessments.tsx
/dpias	DPIAs	scope=TENANT; roles=SUPER_ADMIN,DPO	frontend/src/pages/DPIAs.tsx
/arco	ARCO	scope=TENANT; roles=SUPER_ADMIN,DPO,ADMIN	frontend/src/pages/ARCO.tsx

Ruta	Página	Acceso UI	Archivo
/portability	Portability	scope=TENANT; roles=SUPER_ADMIN,DPO,ADMIN	frontend/src/pages/Portability.tsx
/incidents	Incidents	scope=TENANT; roles=SUPER_ADMIN,DPO,ADMIN,DEPT_HEAD	frontend/src/pages/Incidents.tsx
/consents	Consents	scope=TENANT; roles=SUPER_ADMIN,DPO,ADMIN	frontend/src/pages/Consents.tsx
/legal-documents	LegalDocuments	scope=TENANT; roles=SUPER_ADMIN,DPO,ADMIN	frontend/src/pages/LegalDocuments.tsx
/ropa	ROPA	scope=TENANT; roles=SUPER_ADMIN,DPO,ADMIN,AUDITOR	frontend/src/pages/ROPA.tsx
/action-plans	ActionPlans	scope=TENANT; roles=SUPER_ADMIN,DPO,ADMIN,DEPT_HEAD	frontend/src/pages/ActionPlans.tsx
/audit-plans	AuditPlans	scope=TENANT; roles=SUPER_ADMIN,DPO,AUDITOR	frontend/src/pages/AuditPlans.tsx
/remediations	Remediations	scope=TENANT; roles=SUPER_ADMIN,DPO,ADMIN,DEPT_HEAD	frontend/src/pages/Remediations.tsx
/audit-log	AuditLog	scope=TENANT; roles=SUPER_ADMIN,DPO,AUDITOR	frontend/src/pages/AuditLog.tsx
/training	Training	scope=TENANT	frontend/src/pages/Training.tsx
/backups	Backups	scope=TENANT; roles=SUPER_ADMIN	frontend/src/pages/Backups.tsx
/settings	Settings	scope=TENANT/PLATFORM	frontend/src/pages/Settings.tsx

33 rutas verificadas: 2 públicas y 31 protegidas. Las reglas UI no sustituyen la autorización del backend.

10.4 Navegación por dominios

Grupo	Módulos
Overview	platform, dashboard, alerts, reports
Organization	company profile, users, departments, catalogs, sectors, tenants
Data registry	inventory, treatment activities, information assets, retention, import/export
Risk	risk assessments, DPIA
Rights	ARCO, portability, incidents, consents
Documents	legal documents, ROPA
Operations	action plans, audit plans, remediations

Grupo	Módulos
Transversal	audit log, training, backups, settings

10.5 Componentes compartidos

Categoría	Componentes
Layout	AppShell, Sidebar, Topbar, Breadcrumbs
Datos	DataTable, KPICard, Badge, Tabs
Acciones	Button, IconButton, ActionMenu, Modal
Formularios	Input y controles nativos estilizados por página
Estados	LoadingState, EmptyState, ErrorState, ForbiddenState
Marca	BrandMark, DataLegalWordmark, GlassCard, PageHeader

10.6 Capa de servicios y tipos

Dominio	Funciones exportadas	Archivo
actionPlans	7	frontend/src/api/actionPlans.ts
alerts	5	frontend/src/api/alerts.ts
arco	6	frontend/src/api/arco.ts
audit	2	frontend/src/api/audit.ts
auditPlans	8	frontend/src/api/auditPlans.ts
auth	4	frontend/src/api/auth.ts
backups	4	frontend/src/api/backups.ts
catalogs	5	frontend/src/api/catalogs.ts
client	0	frontend/src/api/client.ts
companyProfile	2	frontend/src/api/companyProfile.ts
consents	10	frontend/src/api/consents.ts
dataInventory	1	frontend/src/api/dataInventory.ts
departments	5	frontend/src/api/departments.ts
dpias	6	frontend/src/api/dpias.ts
importExport	3	frontend/src/api/importExport.ts
incidents	7	frontend/src/api/incidents.ts
informationAssets	5	frontend/src/api/informationAssets.ts

Dominio	Funciones exportadas	Archivo
legalDocuments	5	frontend/src/api/legalDocuments.ts
portability	5	frontend/src/api/portability.ts
remediations	4	frontend/src/api/remediations.ts
reports	5	frontend/src/api/reports.ts
retention	9	frontend/src/api/retention.ts
riskAssessments	6	frontend/src/api/riskAssessments.ts
ropa	2	frontend/src/api/ropa.ts
sectors	3	frontend/src/api/sectors.ts
tenants	3	frontend/src/api/tenants.ts
training	14	frontend/src/api/training.ts
treatmentActivities	9	frontend/src/api/treatmentActivities.ts
users	5	frontend/src/api/users.ts

Los tipos se organizan en auth, dataRegistry, documents, operations, organization, rights, risk y transversal, reexportados desde types/index.ts. La separación reduce acoplamiento, aunque no existe generación automática desde OpenAPI.

10.7 Internacionalización

i18next se inicializa con una única lengua: en.

lng y fallbackLng están fijos en en; Settings muestra la preferencia, pero no cambia idioma.

La mayoría del texto visible usa claves; aún hay cadenas directas en algunas páginas/componentes.

Para Ecuador, añadir es.json, selector persistente, traducción de estados y pruebas de desbordamiento visual.

Limitación funcional: Los endpoints de cookie banner/consentimiento requieren JWT y permiso de consents. Si se pretende captar consentimiento desde un sitio público, hace falta diseñar una superficie pública segura o un servicio separado.

Fuentes: frontend/src/i18n/en.json, main.tsx, Settings.tsx, routes/navigation.ts y archivos frontend/src/api/*.ts.

11. Datos de desarrollo, mock y reinicio local

11.1 Controles de seguridad

seed_dev_data solo ejecuta en modos de desarrollo/prueba y cuando SEED_DEV_DATA es true. El dataset completo requiere además SEED MOCK_DATA. Las funciones usan consultas get-or-create y rellenan campos faltantes, por lo que el proceso busca ser idempotente.

Regla: Usar el seed únicamente sobre una base local descartable. Sustituir las credenciales de muestra mediante variables propias y no reutilizarlas fuera del entorno de desarrollo.

11.2 Contenido del dataset mock

Grupo	Datos
Organización	tenant demo, perfil DPO, cinco departamentos y usuarios con roles
Catálogos	tipos de datos, activos, formatos, medios, clasificación y bases legales
Registro	actividades de tratamiento, activos y políticas/registros de retención
Riesgo	evaluaciones, DPIA, remediaciones y planes de acción
Derechos/seguridad	ARCO, portabilidad e incidentes con estados y fechas
Cumplimiento	consentimientos, documentos, auditoría, capacitación y alertas
Operación	logs de auditoría, respaldos de demostración y logs de retención

Fuente: `backend/app/db/seed_dev.py`. El valor `DEV MOCK DATASET` se lee, pero el código actual siempre carga el conjunto medio completo.

11.3 Ejecutar y repetir el seed local

17. Confirmar `ENVIRONMENT=development` y que la base elegida sea local o desechable.
18. Definir `SEED_DEV_DATA=true`; añadir `SEED MOCK DATA=true` para el conjunto funcional completo.
19. Ejecutar `python -m app.db.seed_dev` desde backend con el entorno virtual activo.
20. Repetir el comando cuando sea necesario: `get-or-create` evita duplicar las entidades base.
21. Para empezar de cero, detener procesos y reemplazar la base SQLite o eliminar deliberadamente el volumen Compose.

```
cd backend
source .venv/bin/activate
SEED_DEV_DATA=true SEED MOCK DATA=true python -m app.db.seed_dev

# reinicio recuperable de SQLite: conservar copia antes de recrear
test -f datalegal.db && mv datalegal.db datalegal.db.bak
alembic upgrade head
SEED_DEV_DATA=true SEED MOCK DATA=true python -m app.db.seed_dev
```

Fuente: `backend/app/db/seed_dev.py`. El documento omite deliberadamente valores de credenciales de desarrollo.

12. Referencia del entorno local

12.1 Topología con Docker Compose

Topología local con Docker Compose

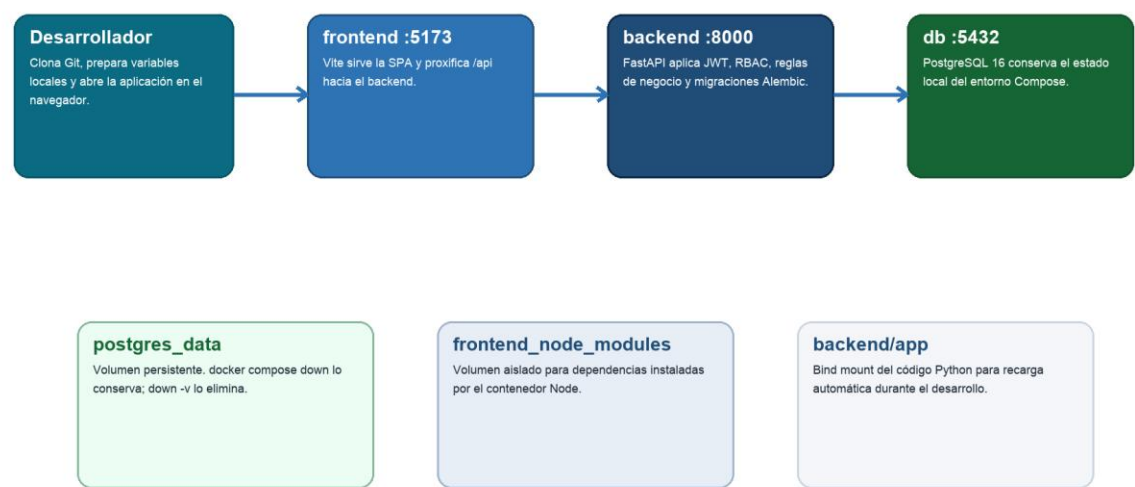


Figura 3. Servicios, puertos y persistencia del entorno local con Docker Compose.

Compose crea una red privada para los tres servicios. El navegador llega a Vite por localhost:5173; Vite reenvía /api al backend; FastAPI usa el nombre de servicio db para conectarse a PostgreSQL. Los puertos publicados existen para diagnóstico local.

12.2 Servicios definidos

Servicio	Imagen/build	Puerto	Persistencia	Arranque
db	postgres:16-alpine	5432	postgres_data	pg_isready
backend	build ./backend	8000	app montada	depende de db healthy
frontend	node:20-alpine	5173	repo + node_modules	depende de backend

El Dockerfile backend parte de python:3.12-slim, instala libpq-dev/gcc, copia pyproject, Alembic, app y scripts, instala el paquete editable y ejecuta migraciones antes de Uvicorn. No define USER no-root ni healthcheck dentro de la imagen.

12.3 Orden de arranque y flujo de datos

22. db inicia, crea o reutiliza postgres_data y pasa pg_isready.

23. backend espera db healthy, aplica alembic upgrade head y ejecuta el seed habilitado.
24. Uvicorn escucha en 8000 con reload; ./backend/app se monta para reflejar cambios Python.
25. frontend ejecuta npm ci, inicia Vite en 5173 y conserva node_modules en volumen separado.
26. Las llamadas /api salen del navegador hacia Vite y se proxifican a backend:8000.

Efecto del estado persistente: Reiniciar contenedores no recrea usuarios ni borra la base. Si un cambio de migración o seed parece no aplicarse, inspeccionar alembic current y decidir conscientemente si se conserva postgres_data o se reinicia el entorno descartable.

12.4 Comandos cotidianos de Compose

Acción	Comando	Cuándo
Construir/iniciar	docker compose up --build	primer arranque o cambios de imagen
Segundo plano	docker compose up -d --build	deja terminal libre
Estado	docker compose ps	salud y puertos
Logs	docker compose logs -f backend frontend db	diagnóstico continuo
Shell backend	docker compose exec backend sh	Alembic o inspección del contenedor
Migración	docker compose exec backend alembic current	revisión aplicada
Detener	docker compose down	conserva volúmenes
Reinicio total	docker compose down -v	destruye datos locales y dependencias del volumen

12.5 Equivalencia de procesos manuales

Responsabilidad	Proceso local	Nota
db	SQLite local por defecto	DATABASE_URL opcional para PostgreSQL local
backend	uvicorn app.main:app --reload	terminal con venv activo
migraciones	alembic upgrade head	antes de seed y servidor
seed	python -m app.db.seed_dev	con flags SEED_* explícitos
frontend	npm run dev	proxy Vite hacia localhost:8000
jobs	python scripts/scan_critical_alerts.py	ejecución manual bajo demanda

Fuentes: docker-compose.yml, backend/Dockerfile, backend/app/db/session.py, backend/app/db/seed_dev.py y frontend/vite.config.ts.

13. Configuración local y variables de entorno

La siguiente matriz documenta nombres y propósito sin revelar valores. `backend/.env` se lee cuando el proceso se inicia desde backend; el `.env` de la raíz sirve para la interpolación de Docker Compose. Ambos deben permanecer fuera de Git.

Variable	Área	Propósito	Sensible
ENVIRONMENT	core	usar development o test en este recorrido local	no
DATABASE_URL	core	SQLite local o URL SQLAlchemy de PostgreSQL local	sí
SECRET_KEY	auth	firma JWT; generar valor local propio	sí
ALGORITHM	auth	algoritmo JWT; código espera HS256	no
ACCESS_TOKEN_EXPIRE_MINUTES	auth	expiración JWT e inactividad	no
MFA_TOKEN_EXPIRE_MINUTES	auth	vigencia mfa_pending	no
MAX_FAILED_ATTEMPTS	auth	umbral de bloqueo	no
LOCKOUT_MINUTES	auth	duración de bloqueo	no
MFA_ENCRYPTION_KEY	MFA	Fernet para secretos TOTP	sí
CORS_ORIGINS	HTTP	allow-list CSV si se restringen orígenes locales	no
TRUSTED_PROXIES	HTTP	CIDR que puede suministrar X-Forwarded-For	no
AUTH_RATE_LIMIT	HTTP	límite de login, sintaxis slowapi	no
MFA_RATE_LIMIT	HTTP	límite de verificación MFA	no
SHOW_DOCS	API	Swagger/ReDoc/OpenAPI	no
SEED_DEV_DATA	seed	habilita el seed de desarrollo	no
SEED MOCK_DATA	seed	habilita dataset coherente	no
DEV MOCK_DATASET	seed	selector nominal; actualmente no cambia volumen	no
DEV_ADMIN_EMAIL	seed	cuenta de desarrollo	dato personal
DEV_ADMIN_PASSWORD	seed	contraseña de desarrollo	sí
DEV_PLATFORM_OWNER_*	seed	cuenta PLATFORM de desarrollo	sí/parcial
VITE_API_PROXY_TARGET	frontend dev	destino del proxy /api	no

Higiene del clon: Antes de compartir una rama, revisar git status y git ls-files '*.env*'. Solo deben publicarse plantillas sin valores sensibles. No copiar la salida de comandos que generan claves a documentación, tickets, capturas o mensajes.

13.1 Generar secretos locales sin fijarlos en el documento

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
# pegar la salida únicamente en el .env local ignorado por Git
git check-ignore .env backend/.env
```

Fuentes: *backend/app/core/config.py*, *backend/app/db/seed_dev.py*, *docker-compose.yml*, *frontend/vite.config.ts* y *backend/.env.example*.

14. Instalación y despliegue en AWS

Este capítulo documenta la implementación AWS observada en el workspace y sirve como procedimiento de instalación para un clon del repositorio. Separa el estado confirmado, los valores que cada organización debe parametrizar y las correcciones necesarias para que una tercera persona pueda repetir el despliegue sin recibir archivos locales.

Arquitectura confirmada: CloudFront, no Cloudflare: La URL pública pertenece a Amazon CloudFront. En esta implementación CloudFront no usa un bucket S3: su origen es el DNS público de una instancia EC2 y Nginx sirve frontend/dist desde esa instancia. No crear S3 ni Cloudflare para reproducir el estado actual.

14.1 Topología y estado observable

Topología AWS implementada

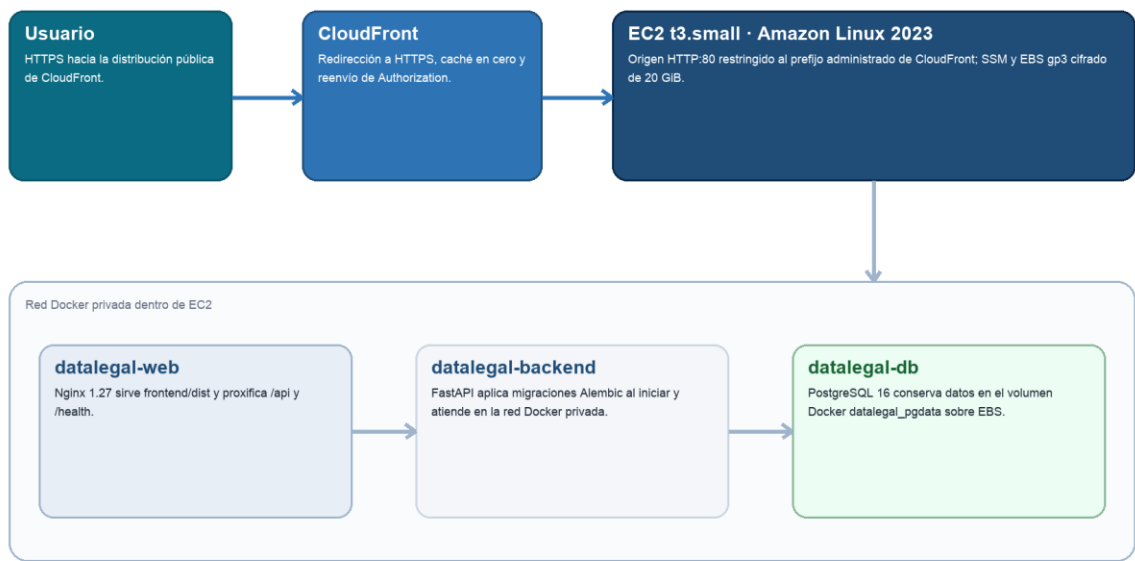


Figura 4. Topología AWS confirmada por el script de despliegue y los endpoints públicos.

Capa	Recurso	Responsabilidad
Ingreso	CloudFront	HTTPS público, redirección HTTP→HTTPS, TTL 0 y Authorization reenviado
Origen	EC2 t3.small	Amazon Linux 2023; HTTP:80 solo desde el prefix list administrado de CloudFront
Web	datalegal-web	Nginx 1.27 sirve frontend/dist y proxifica /api y /health
Aplicación	datalegal-backend	FastAPI; migraciones Alembic en el arranque; no publica el puerto 8000

Capa	Recurso	Responsabilidad
Datos	datalegal-db	PostgreSQL 16 en volumen Docker datalegal_pgdata sobre el EBS raíz cifrado
Administración	AWS Systems Manager	perfil de instancia con AmazonSSMManagedInstanceCore; SSH no se abre

Elemento	Valor/resultado	Nivel de confirmación
Región de cómputo	us-east-2	constante REGION del script
Instancia registrada	i-030a8f1eee350ca02	registrada en docs/deployment-aws.md; revalidar en la consola antes de operar
Distribución registrada	E1N9L2Z4SD2QTM	registrada en docs/deployment-aws.md; CloudFront es un servicio global
URL pública	https://d3ca7tz8eyjv8d.cloudfront.net	observable y activa
/ y /login	HTTP 200	verificación externa del 14-08-2026
/health	HTTP 200	status=ok, version=0.1.0, environment=production
/api/docs	HTTP 404	Swagger deshabilitado en producción, resultado esperado

Fuentes: `deploy/aws/deploy-datalegal.sh`, `docs/deployment-aws.md` y comprobaciones HTTP contra la URL CloudFront el 14-08-2026. Los IDs deben confirmarse en AWS antes de cambios destructivos.

14.2 Prerrequisitos y permisos

Categoría	Requisito
Código	clon de https://github.com/Jonthz/datalegal con el script <code>deploy/aws</code> incluido y revisado
AWS	cuenta con facturación habilitada, cuotas disponibles y región us-east-2 seleccionada
Red	VPC predeterminada y al menos una subred predeterminada con DNS/IP públicos
Herramientas	AWS CloudShell o AWS CLI v2 autenticado; Git y Bash
IAM	permisos para EC2, VPC/seguridad, IAM roles/profiles, SSM y CloudFront
Operación	responsable autorizado para valores de producción, bootstrap y aceptación
Continuidad	ventana de cambio y respaldo verificado antes de actualizar una instalación existente

Bloqueo de reproducibilidad actual: En el commit base 291f533, `deploy/aws` y `docs/deployment-aws.md` aparecen como archivos locales no rastreados; además hay una migración merge pendiente y `.env.render` está rastreado. Antes de entregar solo la URL Git, revisar y versionar los artefactos de despliegue y migración, y retirar/rotar cualquier valor sensible rastreado.

14.3 Variables de producción, sin valores secretos

Variable	Valor conceptual	Finalidad	Sensible
ENVIRONMENT	production	activa validaciones estrictas	no
DATABASE_URL	PostgreSQL interno	datalegal-db:5432; usuario, base y clave parametrizados	sí
SECRET_KEY	generado por instalación	firma JWT; mínimo 32 caracteres	sí
MFA_ENCRYPTION_KEY	Fernet generado	cifra secretos TOTP; debe conservarse entre reinicios	sí
CORS_ORIGINS	URL CloudFront final	allow-list explícita; aunque SPA/API comparten origen	no
SHOW_DOCS	false	oculta Swagger, ReDoc y OpenAPI	no
SEED_DEV_DATA	false	impide datos de desarrollo	no
SEED MOCK_DATA	false	impide dataset mock	no
ALGORITHM	HS256	algoritmo JWT esperado por el código	no
ACCESS_TOKEN_EXPIRE_MINUTES	política del operador	30 en el script observado	no
TRUSTED_PROXIES	CIDR validado	solo si se decide confiar en X-Forwarded-For de Nginx	no

El script genera DATABASE_URL, SECRET_KEY y MFA_ENCRYPTION_KEY en la instancia y escribe /opt/datalegal/backend.env con permisos root-only.

El script usa inicialmente CORS_ORIGINS=https://placeholder.invalid; después de conocer el dominio CloudFront, reemplazarlo por la URL final y recrear el contenedor backend.

Nunca copiar backend.env al repositorio, a la documentación ni a un ticket. Conservar las claves en el mecanismo de secretos aprobado por la organización.

14.4 Preparar el clon que se va a desplegar

27. Clonar main, registrar el commit exacto y confirmar que el árbol de trabajo está limpio.
28. Comprobar que deploy/aws/deploy-datalegal.sh y docs/deployment-aws.md están rastreados en Git; si no, detenerse y publicar una revisión aprobada.
29. Confirmar que alembic heads devuelve una sola revisión y que la migración merge está en el commit.
30. Confirmar que el frontend tiene lockfile versionado y que npm ci, TypeScript y Vite completan.
31. Ejecutar pytest, build, secret scan y git diff --check antes de crear infraestructura.
32. Revisar el script Bash y su diff; no ejecutarlo desde una rama o fork desconocido.

```
git clone https://github.com/Jonthz/datalegal.git
cd datalegal
git branch --show-current
git rev-parse --short HEAD
git status --short
git ls-files --error-unmatch deploy/aws/deploy-datalegal.sh
git ls-files --error-unmatch docs/deployment-aws.md
bash -n deploy/aws/deploy-datalegal.sh
cd backend && alembic heads && cd ..
```

No usar el archivo heredado de otro proveedor: .env.render corresponde a una arquitectura anterior y no es entrada del despliegue AWS. Si contuvo una credencial real, rotarla en el proveedor, retirarla del historial según el procedimiento del equipo y conservar solo .env.render.example.

14.5 Crear o reutilizar la infraestructura desde CloudShell

El helper es idempotente para recursos etiquetados Project=DataLegal y ManagedBy=Codex. En una cuenta nueva crea el grupo de seguridad, rol/perfil SSM, EC2, EBS y CloudFront. En una cuenta con recursos existentes los reutiliza; eso no equivale a actualizar la aplicación.

```
# AWS CloudShell: seleccionar us-east-2 antes de comenzar
export DATALEGAL_REGION=us-east-2
aws ec2 describe-vpcs --region "$DATALEGAL_REGION" \
  --filters Name=is-default,Values=true \
  --query 'Vpcs[0].VpcId' --output text
bash deploy/aws/deploy-datalegal.sh | tee datalegal-deploy-summary.txt
# guardar solo IDs y URL; no copiar archivos env ni salida con secretos
```

Recurso	Nombre/configuración	Comportamiento del script
Security group	datalegal-web-sg	TCP/80 solo desde com.amazonaws.global.cloudfront.origin-facing
IAM role/profile	datalegal-ec2-ssm-role / datalegal-ec2-profile	AmazonSSMManagedInstanceCore
EC2	t3.small	Amazon Linux 2023, root EBS gp3 20 GiB cifrado, DeleteOnTermination=true
CloudFront	comentario DataLegal MVP	custom origin EC2 por HTTP; viewer HTTPS; PriceClass_100; caché desactivada
SSM Parameter	/datalegal/deployment/url	URL pública, tipo String

Costo y estabilidad: EC2, EBS y CloudFront generan cargos. El script no crea presupuesto, alarma, Elastic IP, ALB ni un volumen de datos separado. Detener/iniciar la EC2 puede cambiar su DNS público y dejar el origen CloudFront apuntando al valor anterior; validar el origen después de cada cambio de estado.

14.6 Esperar el bootstrap de infraestructura y revisar la instancia

33. Copiar INSTANCE_ID, CLOUDFRONT_ID y URL impresos por el script en el registro de cambio.
34. Esperar que EC2 figure running y que Systems Manager muestre el nodo como Online.
35. Abrir una sesión SSM; no habilitar SSH ni ampliar el security group para diagnosticar.
36. Revisar /var/log/datalegal-deploy.log y confirmar /opt/datalegal/DEPLOYED.
37. Confirmar tres contenedores activos y la salud de PostgreSQL, FastAPI y Nginx.

```
export DATALEGAL_REGION=us-east-2
export DATALEGAL_INSTANCE_ID='<instance-id-impreso>'
aws ec2 wait instance-status-ok --region "$DATALEGAL_REGION" \
  --instance-ids "$DATALEGAL_INSTANCE_ID"
aws ssm describe-instance-information --region "$DATALEGAL_REGION" \
  --filters Key=InstanceIds,Values="$DATALEGAL_INSTANCE_ID"
aws ssm start-session --region "$DATALEGAL_REGION" \
  --target "$DATALEGAL_INSTANCE_ID"

# dentro de la sesión SSM
sudo test -f /opt/datalegal/DEPLOYED
sudo tail -n 120 /var/log/datalegal-deploy.log
sudo docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
sudo docker exec datalegal-db pg_isready -U datalegal -d datalegal
```

14.7 Completar CORS y crear el tenant administrador

La infraestructura no equivale al alta funcional. El script actual no ejecuta backend/app/db/bootstrap_prod.py ni crea los parámetros de email/contraseña que menciona docs/deployment-aws.md. El operador debe completar una sola vez los pasos siguientes.

38. En SSM, sustituir CORS_ORIGINS por la URL CloudFront final dentro de backend.env.
39. Recrear datalegal-backend con el mismo network y env-file; un docker restart no vuelve a leer el archivo.
40. Crear un archivo temporal root-only con las cinco variables BOOTSTRAP obligatorias; introducir valores autorizados sin pegarlos en el historial.
41. Ejecutar python -m app.db.bootstrap_prod dentro del backend y confirmar el tenant/admin creado.
42. Eliminar el archivo temporal, conservar la contraseña solo en el gestor de secretos autorizado y probar login.

Variable temporal	Contenido esperado	Sensible
BOOTSTRAP_TENANT_NAME	nombre legal o comercial del primer tenant	no
BOOTSTRAP_TENANT_RUC	identificador único aprobado	dato organizacional
BOOTSTRAP_ADMIN_EMAIL	cuenta nominativa de administración	dato personal
BOOTSTRAP_ADMIN_PASSWORD	contraseña fuerte y única	sí

Variable temporal	Contenido esperado	Sensible
BOOTSTRAP_ADMIN_NAME	nombre de la persona administradora	dato personal
BOOTSTRAP_TENANT_COUNTRY	opcional; Ecuador por defecto	no
BOOTSTRAP_TENANT_SECTOR	opcional	no

```
# dentro de SSM: actualizar solo CORS_ORIGINS y recrear backend
sudo -i
vi /opt/datalegal/backend.env
grep '^CORS_ORIGINS=' /opt/datalegal/backend.env
docker rm -f datalegal-backend
docker run -d --name datalegal-backend --restart unless-stopped \
  --network datalegal --env-file /opt/datalegal/backend.env \
  datalegal-backend:latest

# crear bootstrap temporal; no escribir secretos directamente en el historial
umask 077
install -m 600 /dev/null /opt/datalegal/bootstrap.env
vi /opt/datalegal/bootstrap.env
docker exec --env-file /opt/datalegal/bootstrap.env \
  datalegal-backend python -m app.db.bootstrap_prod
rm -f /opt/datalegal/bootstrap.env
```

Idempotencia del bootstrap: bootstrap_prod reutiliza el tenant por RUC y el usuario por email. Si el email ya existe con un rol diferente de SUPER_ADMIN, se detiene y no lo modifica. No habilitar SEED_DEV_DATA ni SEED MOCK_DATA en producción para resolver un alta faltante.

14.8 Verificación y criterios de aceptación

Control	Ejecución	Esperado
HTTPS	curl -I <URL>/login	200; certificado válido y sin redirección a HTTP
Salud	curl -fsS <URL>/health	status=ok y environment=production
Documentación API	curl -o /dev/null -w '%{http_code}' <URL>/api/docs	404
Origen	security group EC2	sin 0.0.0.0/0; TCP/80 solo prefix list CloudFront
Contenedores	docker ps / logs	web, backend y db activos; sin bucles de reinicio
Migraciones	docker exec datalegal-backend alembic current	revisión head esperada
Base	pg_isready y consulta funcional	PostgreSQL acepta conexión interna
Login	interfaz con cuenta bootstrap	acceso correcto y cambio/control de credencial según política
Datos mock	consulta visual	ausentes salvo aprobación explícita; flags SEED_* en false
Persistencia	reinicio controlado de backend/web	usuarios y datos continúan en datalegal_pgdata

```
export DATALEGAL_URL='https://d3ca7tz8eyjv8d.cloudfront.net'
curl -fsS "$DATALEGAL_URL/health"
curl -sS -o /dev/null -w 'login=%{http_code}\n' "$DATALEGAL_URL/login"
curl -sS -o /dev/null -w 'docs=%{http_code}\n' "$DATALEGAL_URL/api/docs"
```


14.9 Actualizar una instalación existente

El script no publica una nueva versión: Cuando encuentra la EC2 etiquetada, deploy-datalegal.sh solo comprueba/inicia la instancia y reutiliza CloudFront. No ejecuta git pull, no reconstruye imágenes, no aplica un nuevo frontend y no recrea contenedores. Usar una ventana de actualización separada.

43. Abrir cambio, identificar commit objetivo y confirmar QA/migraciones en un entorno no productivo.
44. Registrar commit actual, IDs de imagen y configuración; realizar pg_dump y verificar checksum antes de tocar contenedores.
45. Hacer git fetch y checkout del commit aprobado; no desplegar una rama mutable sin registrar SHA.
46. Construir una imagen backend etiquetada con el SHA y un frontend con lockfile; no sustituir latest hasta validar el build.
47. Ejecutar alembic upgrade head con la nueva imagen sobre la red privada y detenerse ante cualquier error.
48. Recrear backend y web conservando backend.env, red datalegal y volumen datalegal_pgdata.
49. Ejecutar todos los criterios de aceptación; conservar evidencia y el SHA anterior para rollback.

```
# sesión SSM como root
cd /opt/datalegal/repo
export DATALEGAL_PREVIOUS_REV="$(git rev-parse HEAD)"
git fetch --prune origin
git checkout --detach '<sha-aprobado>'
export DATALEGAL_RELEASE_REV="$(git rev-parse --short HEAD)"
docker build -t "datalegal-backend:$DATALEGAL_RELEASE_REV" ./backend
docker run --rm -v "$PWD/frontend:/app" -w /app node:20-alpine \
  sh -c 'npm ci --no-audit --no-fund && npm run build'
docker run --rm --network datalegal --env-file /opt/datalegal/backend.env \
  "datalegal-backend:$DATALEGAL_RELEASE_REV" alembic upgrade head

# recrear procesos stateless; datalegal-db y datalegal_pgdata permanecen
docker rm -f datalegal-web datalegal-backend
docker run -d --name datalegal-backend --restart unless-stopped \
  --network datalegal --env-file /opt/datalegal/backend.env \
  "datalegal-backend:$DATALEGAL_RELEASE_REV"
for DATALEGAL_ATTEMPT in $(seq 1 60); do
  docker exec datalegal-backend python -c \
    "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8000/health').read()" \
    >/dev/null 2>&1 && break
  sleep 3
done
docker exec datalegal-backend python -c \
  "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8000/health').read()"
docker run -d --name datalegal-web --restart unless-stopped \
  --network datalegal -p 80:80 \
  -v /opt/datalegal/repo/frontend/dist:/usr/share/nginx/html:ro \
  -v /opt/datalegal/repo/deploy/aws/nginx.conf:/etc/nginx/nginx.conf:ro \
  nginx:1.27-alpine
```

La recreación final de los contenedores debe conservar exactamente los mounts, el network, la política restart y los nombres definidos por el script. Revisar el diff del helper antes de copiar comandos. CloudFront usa TTL 0 en la configuración actual, por lo que no requiere invalidación para cada build; reevaluarlo si se habilita caché.

14.10 Rollback y recuperación

Capa	Acción	Condición
Frontend/backend	reconstruir o reutilizar imagen/artefacto del SHA anterior y recrear web/backend	base compatible con la versión anterior
Migración compatible	preferir forward-fix y conservar la nueva revisión	error funcional sin corrupción de datos
Base de datos	declarar mantenimiento, detener escrituras y restaurar el dump verificado	solo con autorización y plan de pérdida de datos
CloudFront	restaurar el origen/config anterior mediante versión registrada	cambio accidental de distribución
EC2	corregir/recrear desde respaldo; no terminar la instancia de forma improvisada	EBS raíz usa DeleteOnTermination=true

No ejecutar alembic downgrade sin revisar que la migración sea reversible y que no destruya datos.

Un volumen Docker no es un backup independiente: permanece en el mismo EBS y se pierde si termina la instancia.

Registrar hora, versión, RPO aceptado, dump usado, checksum y resultado de smoke tests.

El procedimiento detallado de dump/restauración debe mantenerse alineado con el Manual de base de datos y respaldo.

14.11 Solución de problemas del despliegue

Síntoma	Causa probable	Acción segura
El script no existe tras clonar	deploy/aws no fue versionado	usar un commit aprobado que lo incluya; no copiar un script de chat
No hay VPC/subred	la cuenta no conserva red predeterminada	adaptar el helper a IDs de red revisados; no crear reglas abiertas
EC2 sin PublicDnsName	subred sin IP pública o instancia aún no lista	revisar route table, auto-assign y estado; el origen necesita DNS alcanzable
SSM Offline	rol/profile, agente o salida de red	revisar instance profile, SSM Agent y conectividad; no abrir SSH como atajo
Falta DEPLOYED	user-data falló	leer /var/log/datalegal-deploy.log desde el primer error
CloudFront 502/504	Nginx caído, DNS de origen obsoleto o puerto 80 inaccesible	docker ps/logs, origen y security group
CloudFront 403	comportamiento/origen o regla prefix list incorrectos	revisar distribución y SG sin ampliar a Internet

Síntoma	Causa probable	Acción segura
Login carga pero no autentica	bootstrap ausente, bloqueo o DB/migración	crear cuenta con bootstrap_prod o diagnosticar auth; no activar seed mock
CORS en una integración	placeholder o allow-list incompleta	actualizar CORS_ORIGINS y recrear backend
Rerun sin cambios visibles	el helper reutiliza la EC2 sin actualizar código	ejecutar el procedimiento 14.9
Falla tras stop/start	DNS público de EC2 cambió	actualizar origin CloudFront o adoptar EIP/ALB tras diseño y aprobación
Backend reinicia	Alembic, env o DB no listos	docker logs datalegal-backend; validar un head y variables requeridas

Fuentes: `deploy/aws/deploy-datalegal.sh`, `backend/Dockerfile`, `backend/app/core/config.py`, `backend/app/db/bootstrap_prod.py`, `backend/alembic` y comprobación de los endpoints públicos. Recomendaciones como EIP/ALB, presupuestos o un volumen de datos separado no están implementadas.

15. Pruebas, build y controles de calidad

15.1 Resultado ejecutado

Control	Resultado	Detalle
Backend pytest	871/871 aprobadas	90,45 s sin cobertura / 94,05 s con cobertura
Cobertura app	93 %	4.172 statements; 300 no cubiertos
Warnings	35	crypt Python 3.13, HTTP 422 y API pdf2 In
TypeScript	sin errores	tsc --noEmit
Build frontend	exitoso	1.978 módulos transformados
Alembic en workspace	un head tras merge local	el commit base conserva dos heads y requiere corrección

```
# backend
cd backend
PYTHONPATH=. pytest -q --cov=app --cov-report=term

# frontend
cd frontend
npm run lint
npm run build

# migraciones
cd backend
alembic heads
```

Ejecución local del 09-08-2026. En el workspace revisado fue necesario `PYTHONPATH=.` porque el venv existente no tenía app instalada editable.

15.2 Cobertura por áreas

Nivel	Módulos representativos
Excelente (>=95 %)	reports 99, incidents 95, legal_documents 97, sectors 98, middleware 95, SLA 100
Alta (85–94 %)	auth 88, users 94, roles 94, DPIA 95, backups 90, critical_alerts 94
Reforzar (<85 %)	departments 58, rate_limit 50, remediations 76, risk_assessments 79, deps 83

15.3 Organización de la suite

Archivo	Funciones test	Líneas
test_alembic_baseline.py	2	66
test_audit.py	9	97
test_auth.py	14	204
test_catalogs.py	7	94
test_critical_alerts.py	13	231
test_data_inventory.py	8	191
test_inactivity.py	3	32
test_incidents.py	23	311
test_information_assets.py	10	154
test_mfa_crypto.py	6	91
test_permissions.py	11	111
test_portability.py	8	130
test_rate_limit.py	3	80
test_retention.py	11	196
test_risk_assessments.py	10	170
test_risk_scoring_unit.py	17	231
test_roles.py	18	239
test_security_hardening.py	9	133
test_sprint3.py	45	639
test_sprint4.py	43	675
test_sprint5.py	41	534
test_tenants.py	10	154
test_training.py	13	225
test_treatment_activities.py	11	163

Archivo	Funciones test	Líneas
test_users.py	9	116

Dato desactualizado: README.md declara 268 pruebas y ~92 %; la ejecución actual recolectó 871 y obtuvo 93 %. Actualizar el README para evitar confusión durante la entrega.

15.4 CI actual y recomendado

El único workflow GitHub encontrado instala dependencias manualmente y ejecuta pylint backend/. No ejecuta pytest, coverage, Alembic, TypeScript ni Vite.

Añadir job backend: instalación bloqueada, ruff/pylint, alembic upgrade sobre PostgreSQL efímero, pytest y umbral de cobertura.

Añadir job frontend: npm ci, npm run lint y npm run build; después incorporar tests unitarios y e2e.

Añadir escaneo de secretos/dependencias e imágenes; bloquear merge ante hallazgos críticos.

Publicar artefactos de cobertura y reportes, y exigir los checks en protección de main.

Fuente: [.github/workflows/pylint.yml](#).

16. Diagnóstico, jobs y respaldos

16.1 Señales disponibles

Señal	Contenido	Uso
/health	status/version/environment	confirmar que la API local o CloudFront responde
AuditLog	actor, acción, recurso, detalle, IP, timestamp	trazabilidad funcional
Docker logs	stdout/stderr por contenedor	diagnóstico operativo
Uvicorn/Vite	logs de request, reload y compilación	diagnóstico en ejecución manual
CloudFront	status HTTP, dominio y origen	disponibilidad del ingreso público
EC2/SSM	estado, system status y sesión administrada	diagnóstico AWS sin SSH
user-data	/var/log/datalegal-deploy.log y marcador DEPLOYED	instalación inicial
Alertas internas	ARCO, auditoría, brechas, tareas	operación de cumplimiento
Reportes KPI	riesgos, ARCO, incidentes, planes	visibilidad del tenant

Brecha: No se encontró logging estructurado central, métricas Prometheus, tracing, Sentry, ni correlación request-id. /health no valida conectividad DB ni estado de migraciones, por lo que debe complementarse con alembic current y una consulta funcional.

16.2 Escaneo de alertas críticas

services/critical_alerts.py revisa solicitudes ARCO abiertas y auditorías planificadas próximas.

Genera ARCO_OVERDUE, ARCO_DEADLINE_APPROACHING y AUDIT_SCHEDULED.

El control anti-spam limita a dos alertas por recurso+tipo y día UTC.

scripts/scan_critical_alerts.py recorre tenants activos y es apto para cron.

Docker Compose no programa el job; para desarrollo se ejecuta manualmente cuando se necesite probar alertas.

```
cd backend
source .venv/bin/activate
python scripts/scan_critical_alerts.py
```

16.3 Respallos del entorno local y AWS

scripts/backup.py copia SQLite, calcula SHA-256, verifica inmediatamente y conserva un número configurable de archivos.

El endpoint /backups/create realiza snapshot SQLite solo para SUPER_ADMIN porque contiene todos los tenants.

Con PostgreSQL, el endpoint crea un registro PENDING y pide ejecutar pg_dump manualmente; no produce el archivo.

El volumen postgres_data sobrevive a docker compose down, pero no reemplaza una copia de seguridad.

Para conservar una base Compose, ejecutar un pg_dump local y probar su restauración antes de usar down -v.

En AWS, datalegal_pgdata reside sobre el EBS raíz de la EC2; DeleteOnTermination=true exige copias externas verificadas.

Antes de cada release AWS, generar pg_dump, checksum y prueba de restauración conforme al manual de respaldo.

Entorno descartable: Si los datos son solo mock, el procedimiento más simple es recrear el volumen y volver a ejecutar migraciones + seed. Si se introdujeron casos manuales valiosos, respaldarlos antes de eliminar la base.

Fuentes: backend/app/services/critical_alerts.py, backend/scripts/scan_critical_alerts.py, backend/scripts/backup.py, backend/app/api/v1/backups.py, docker-compose.yml y deploy/aws/deploy-datalegal.sh.

17. Seguridad y riesgos conocidos

Esta sección no sustituye una prueba de penetración. Resume controles observados y riesgos concretos identificables por inspección del código/configuración.

17.1 Controles implementados

Dominio	Control observado
Identidad	bcrypt, JWT firmado, usuario activo, lockout, inactividad y MFA TOTP
MFA	secreto Fernet en reposo y token pendiente de corta duración
Autorización	RBAC backend, roles custom, scopes TENANT/PLATFORM y filtros tenant
HTTP	CORS configurable y cabeceras defensivas para respuestas API
Abuso	rate limit en login/MFA y confianza explícita de proxy
Auditoría	log append-only convencional y registro de fallos de permiso
Configuración	Settings tipado y backend.env root-only en la EC2

Dominio	Control observado
Datos locales	SQLite descartable o PostgreSQL Compose con volumen separado
Ingreso AWS	CloudFront HTTPS y puerto EC2 80 restringido al prefix list administrado
Administración AWS	SSM sin SSH público; EBS raíz cifrado

17.2 Registro de riesgos

ID	Sev.	Hallazgo	Tratamiento
R-01	Crítico	.env.render rastreado con una credencial heredada	rotar en el proveedor, retirar del historial y conservar solo plantilla
R-02	Alto	Dos heads Alembic en el commit base	versionar merge revision y probar clon limpio
R-03	Alto	Sin lockfile frontend versionado	versionar package-lock y mantener npm ci
R-13	Alto	Artefactos deploy/aws no rastreados en el commit base	revisar, versionar y desplegar solo un SHA aprobado
R-14	Alto	Base AWS vive en EBS raíz con DeleteOnTermination=true	backups externos, restauración probada y protección de terminación
R-15	Alto	Origen CloudFront usa DNS público mutable de EC2	revalidar tras stop/start o diseñar EIP/ALB
R-16	Alto	Rerun del helper no actualiza código existente	pipeline de release explícito con SHA, backup y recreación
R-04	Alto	JWT accesible en localStorage	cookies HttpOnly o arquitectura de tokens endurecida
R-05	Alto	Aislamiento tenant solo en aplicación	controles de referencia y pruebas negativas sistemáticas
R-06	Medio	slowapi usa memoria local	documentar límite local y abstraer store si cambia el modelo de ejecución
R-07	Medio	Docker backend corre como root	usuario no-root y capabilities mínimas
R-08	Medio	CI solo Pylint	pruebas, build, migraciones, SAST/SCA y secret scan
R-09	Medio	Backup PostgreSQL queda PENDING	pg_dump local verificable o limitar feature a SQLite
R-10	Medio	Health no comprueba DB/migraciones	añadir readiness separado y test funcional
R-11	Bajo	Warnings/deprecaciones	actualizar crypt/http status/fpdf2 antes de upgrades
R-12	Bajo	README y cifras de pruebas desactualizados	actualizar documentación junto con cambios
R-17	Medio	Tramo CloudFront→EC2 usa HTTP	mantener SG restringido y evaluar TLS al origen si cambia el riesgo

Los riesgos son recomendaciones derivadas del estado observado; priorización final corresponde al responsable de seguridad/negocio.

17.3 Checklist de entrega segura del repositorio y AWS

Un clon limpio contiene un solo head Alembic y package-lock.json compatible con npm ci.

Ningún .env con valores, base, backup, token, log o artefacto local aparece en git status.

871+ pruebas, cobertura acordada, lint y build frontend verdes.

README y .env.example explican requisitos y variables sin publicar credenciales utilizables.

Docker Compose y ejecución manual completan migraciones, seed y smoke checks.

Las cuentas y datos mock se usan únicamente en una base local descartable.

Pruebas de aislamiento tenant, roles, lockout, MFA y exportaciones completadas.

deploy/aws y la merge revision están rastreados en el commit aprobado; .env.render real no lo está.

CloudFront apunta al DNS actual de EC2; el security group no permite 0.0.0.0/0 y SSM figura Online.

Existe un pg_dump externo verificado antes de release y se conoce el efecto de DeleteOnTermination.

La cuenta bootstrap fue creada sin seed mock y sus valores viven solo en el gestor de secretos autorizado.

18. Mantenimiento del código y del despliegue

Cadencia	Actividades
Cada clon	registrar commit; revisar README/.env.example; preflight lockfile + Alembic
Cada sesión	git status; salud, logs, puertos y estado de migración
Antes de commit	tests, lint, build, secret scan y revisión del diff
Semanal	actualizar rama, revisar dependencias y limpiar artefactos locales obsoletos
Cada release AWS	backup+checksum; SHA aprobado; build/migración; smoke tests; evidencia y rollback
Tras stop/start EC2	confirmar DNS público, origen CloudFront, SSM y health
Mensual AWS	costos, parches, almacenamiento, backups, restauración y reglas de seguridad
Antes de borrar datos	respaldar casos manuales útiles o confirmar que todo es mock

18.1 Procedimiento recomendado para actualizar una rama

50. Confirmar el commit base y que git status no contiene cambios accidentales ni secretos.
51. Crear o actualizar la rama de trabajo y leer cambios recientes en README, dependencias y migraciones.
52. Respalidar la base local si contiene casos manuales que deban conservarse.
53. Ejecutar backend tests+coverage, lint, frontend lint/build y escaneo de dependencias/secretos.
54. Aplicar migraciones en una base descartable y ejecutar el seed idempotente.
55. Levantar Compose o los procesos manuales y ejecutar smoke tests de roles críticos.
56. Validar login/MFA, dashboard, writes, exportes, health y logs; registrar evidencia.

57. Revisar git diff --check y git status; excluir bases, .env, backups, logs y builds.
58. Crear un commit pequeño y una PR con comandos de QA, cambio de esquema y pasos de reproducción local.

18.2 Dependencias y migraciones

Renovar dependencias en PR separado, con lockfiles y pruebas completas.

Tratar cambios de bcrypt/passlib, jose, cryptography y SQLAlchemy como sensibles.

Mantener migraciones pequeñas, revisables y reversibles cuando sea posible.

No editar una migración ya compartida; crear una revisión correctiva nueva salvo acuerdo explícito del equipo.

Verificar alembic heads en CI para impedir múltiples heads accidentales.

Desplegar migraciones con estrategia compatible hacia atrás cuando el rollback de aplicación sea necesario.

No usar latest como única referencia operativa: etiquetar imágenes y registrar el SHA desplegado.

19. Solución de problemas

Síntoma	Causa probable	Diagnóstico/acción
401 Invalid/expired	token ausente/expirado, usuario inactivo o inactividad	limpiar sesión, relogin; revisar ACCESS_TOKEN_EXPIRE_MINUTES y last_activity_at
423 Account locked	fallos repetidos	esperar LOCKOUT_MINUTES; soporte autorizado revisa failed_attempts/locked_until sin compartir claves
403 Permission denied	rol, custom role, scope o tenant incorrectos	ver /roles, account_scope y auditoría permission_check_fail
422 Validation	payload no coincide con esquema	comparar body/query con Swagger dev o schemas/*.py
429 Too many requests	rate limit auth/MFA	respetar Retry-After; revisar IP real y TRUSTED_PROXIES
CORS en navegador	origen local no permitido	ajustar CORS_ORIGINS y reiniciar backend
MFA falla tras reinicio	MFA_ENCRYPTION_KEY cambió	restaurar la clave local anterior o recrear MFA en datos descartables
Alembic multiple heads	ramas de migración no unidas	alembic heads; crear y revisar una merge revision
npm ci falla	package-lock ausente	generar el lockfile temporal y versionar una solución estable
Backend no inicia	alembic upgrade head es ambiguo	resolver los dos heads del commit base antes de iniciar Compose
Frontend no conecta	Vite apunta a un backend incorrecto	revisar VITE_API_PROXY_TARGET y puerto 8000
Backup PENDING	PostgreSQL no se exporta desde endpoint	ejecutar pipeline pg_dump/snapshot y actualizar registro/proceso
Alertas SLA no aparecen	job no ejecutado	lanzar scripts/scan_critical_alerts.py manualmente

Síntoma	Causa probable	Diagnóstico/acción
/api/docs 404	SHOW_DOCS falso o entorno equivocado	usar development local con SHOW_DOCS=true
Puerto ocupado	otro proceso usa 5173/8000/5432	identificar proceso o cambiar el puerto local
Seed no aparece	flags SEED_* ausentes o base persistente	revisar entorno, logs y ejecutar seed explícitamente
CloudFront 502/504	origen, Nginx o DNS EC2	seguir 14.11; no abrir la EC2 a Internet
Deploy rerun no actualiza	el helper solo reutiliza infraestructura	aplicar el release documentado en 14.9

19.1 Comandos de diagnóstico local

```
# aplicación
curl -i http://localhost:8000/health
curl -i http://localhost:8000/api/docs
curl -I http://localhost:5173

# contenedores
docker compose ps
docker compose logs --tail 200 backend frontend db

# base/migraciones
docker compose exec backend alembic heads
docker compose exec backend alembic current
docker compose exec db pg_isready

# puertos y espacio
ls -l -n -P -iTCP:5173 -iTCP:8000 -iTCP:5432 -sTCP:LISTEN
docker system df
```

Seguridad: No pegar env, tokens, hashes, connection strings ni archivos de backup completos en tickets o chats. Redactar antes de compartir logs.

20. Guía de contribución

20.1 Preparación

59. Clonar <https://github.com/Jonthz/datalegal.git>, registrar commit base y crear rama desde main actualizada.
60. Instalar backend con extras dev y frontend con lockfile versionado.
61. Copiar solo archivos .env.example; generar valores locales y mantener los .env ignorados.
62. Ejecutar migraciones y seed únicamente en base de desarrollo desechable.

20.2 Convenciones de implementación

Router por dominio en backend/app/api/v1 y registro explícito en router.py.

Esquemas Create/Update/Read en schemas; no devolver modelos con campos secretos.

Toda consulta tenant-scoped filtra tenant_id y valida referencias relacionadas.

Toda acción sensible exige require_permission y crea AuditLog cuando corresponda.

Las reglas compartidas viven en core/services, no duplicadas entre routers.

Frontend: servicio tipado en src/api, tipo de dominio en src/types, ruta lazy y navegación con permisos coherentes.

Texto visible nuevo debe entrar a i18n; evitar cadenas hard-coded si habrá español.

20.3 Definición de terminado

Pruebas positivas, negativas, permiso y aislamiento tenant incluidas.

Migración y downgrade/estrategia forward-fix revisados cuando cambia schema.

pytest/cobertura, lint, build y chequeo Alembic aprobados.

Sin secretos, artefactos locales, bases, backups ni logs en el diff.

Documentación de API/operación actualizada y riesgos nuevos explícitos.

PR pequeño, con evidencia de QA e instrucciones de reproducción local.

20.4 Commits y revisión

feat(scope): descripción breve

fix(auth): corrige bloqueo tras intentos fallidos

test(tenancy): cubre referencia cruzada entre tenants

docs(local): aclara clon, configuración y arranque

Recomendación de proceso; el repositorio no contiene CONTRIBUTING.md en el estado revisado.

21. Glosario

Término	Definición
ARCO	Acceso, Rectificación, Cancelación y Oposición; derechos del titular.
DPIA	Evaluación de impacto en protección de datos.
DPO	Delegado de Protección de Datos.
LOPD	Ley Orgánica de Protección de Datos Personales de Ecuador.
ROPA	Registro de Actividades de Tratamiento.
Tenant	Organización lógica aislada dentro del SaaS.
Account scope	Ámbito TENANT o PLATFORM de una cuenta.
RBAC	Control de acceso basado en roles.
TOTP	Código temporal basado en tiempo usado por MFA.

Término	Definición
JWT	Token firmado que transporta identidad y expiración.
Fernet	Cifrado autenticado usado para el secreto TOTP en reposo.
SLA	Plazo de atención; se representa con semáforo en ARCO.
RPO/RTO	Pérdida máxima tolerada / tiempo objetivo de recuperación.
Compose	Herramienta para definir y ejecutar los servicios locales en contenedores.
CloudFront	CDN/ingreso global de AWS que entrega HTTPS y reenvía al origen EC2.
EC2	Instancia de cómputo AWS que hospeda los contenedores en la topología actual.
EBS	Volumen de bloque de EC2; el despliegue usa un root gp3 cifrado de 20 GiB.
SSM	AWS Systems Manager; acceso administrado a la instancia sin exponer SSH.
Custom origin	Origen HTTP/HTTPS de CloudFront que no es un bucket S3.
Alembic head	Extremo actual de una rama de migraciones; el repositorio debe converger en uno.
RLS	Row-Level Security de PostgreSQL.
CSP	Content Security Policy contra carga/ejecución de contenido no autorizado.

22. Apéndice A — módulos y archivos clave

Área	Archivo	Responsabilidad
Entrada API	backend/app/main.py	FastAPI, middleware, router y health
Router agregado	backend/app/api/v1/router.py	registro de los 29 routers
Dependencias	backend/app/api/deps.py	usuario, tenant, permisos y actividad
Configuración	backend/app/core/config.py	Settings y política prod/dev
Seguridad	backend/app/core/security.py	bcrypt, JWT y TOTP
Cifrado MFA	backend/app/core/mfa_crypto.py	Fernet y compatibilidad legacy
RBAC	backend/app/core/permissions.py	matriz de roles y custom roles
Rate limit	backend/app/core/rate_limit.py	IP/proxy y limiter
Headers	backend/app/core/middleware.py	cabeceras HTTP defensivas
SLA	backend/app/core/sla.py	semáforo y plazos
DB base	backend/app/db/base.py	Base, timestamps y TenantBase
Sesión DB	backend/app/db/session.py	engine y SessionLocal
Registro modelos	backend/app/db/init_db.py	imports de metadata
Seed	backend/app/db/seed_dev.py	dataset demo idempotente

Área	Archivo	Responsabilidad
Bootstrap prod	backend/app/db/bootstrap_prod.py	alta idempotente del primer tenant y SUPER_ADMIN
Auditoría	backend/app/models/audit_log.py	log append-only
Usuario	backend/app/models/user.py	identidad, logout, MFA y scope
Roles custom	backend/app/models/role.py	Role/RolePermission
Plataforma	backend/app/models/platform_access.py	permisos PLATFORM
Alertas job	backend/app/services/critical_alerts.py	plazos críticos
Backup CLI	backend/scripts/backup.py	snapshot SQLite
Alertas CLI	backend/scripts/scan_critical_alerts.py	escaneo multi-tenant
Alembic	backend/alembic/env.py	URL y metadata
App frontend	frontend/src/App.tsx	registro de rutas
Rutas	frontend/src/routes/routes.tsx	públicas/protegidas
Navegación	frontend/src/routes/navigation.ts	grupos sidebar
Permisos UI	frontend/src/routes/permissions.ts	rol/scope/plataforma
Auth UI	frontend/src/hooks/useAuth.ts	estado local de sesión
Axios	frontend/src/api/client.ts	base URL e interceptores
Shell	frontend/src/components/layout/AppShell.tsx	layout autenticado
Traducciones	frontend/src/i18n/en.json	catálogo English
Compose	docker-compose.yml	entorno local
Deploy AWS	deploy/aws/deploy-datalegal.sh	EC2, IAM/SSM, SG, CloudFront y contenedores
Notas AWS	docs/deployment-aws.md	topología, recursos registrados y validación
Imagen backend	backend/Dockerfile	build y CMD
Plantilla local	backend/.env.example	nombres de variables sin publicar valores reales
README	README.md	visión funcional, stack y estructura
CI	.github/workflows/pylint.yml	Pylint en push/PR

23. Apéndice B — evidencia y limitaciones de la revisión

Evidencia	Resultado
Código	inspección estática de backend/app, frontend/src, Docker, README y configuración
Inventarios	AST: 156 endpoints, 33 modelos, 101 esquemas, 33 rutas
Backend	pytest 871/871; coverage app 93 %

Evidencia	Resultado
Frontend	tsc --noEmit y build Vite exitosos
Migraciones	commit base con dos heads; workspace verificado después de merge local
Git	main 291f533; URL origin coincide con el repositorio indicado
AWS	helper inspeccionado; /, /login y /health responden; /api/docs permanece oculto

No se ejecutó un pentest ni un escaneo SAST/SCA completo.

No se probaron restauración PostgreSQL, carga, concurrencia, accesibilidad ni compatibilidad multi-browser.

No se incluyeron secretos, contraseñas de muestra ni cadenas de conexión en el documento.

El arranque Compose del commit base requiere resolver el lockfile ausente y los dos heads Alembic.

Los artefactos deploy/aws y docs/deployment-aws.md están presentes en el workspace, pero no rastreados por el commit base.

No se consultó el inventario AWS mediante CLI local; los IDs provienen del registro del despliegue y deben revalidarse en la consola.

No se ejecutaron una restauración PostgreSQL ni pruebas de carga/caos sobre el entorno AWS.

Los resultados de QA corresponden al workspace revisado; repetirlos después de clonar otro commit.

Criterio de actualización: Regenerar este documento cuando cambien routers, modelos, rutas frontend, variables, Docker Compose, deploy/aws, recursos CloudFront/EC2, requisitos o riesgos. Los inventarios pueden volver a extraerse con el script work/tech-doc/inventory.py.